08/11/2025

```
C:\Users\Femi\OneDrive - Technological University Dublin (1)\Desktop\project>npx create-next-app@latest
```

```
Installing dependencies:
- react
- react-dom
- next

Installing devDependencies:
- @tailwindcss/postcss
- tailwindcss
```

MongoDB for database

```
C:\Users\Femi\OneDrive - Technological University Dublin (1)\Desktop\project\my-app>npm install mongodb
(node:21152) ExperimentalWarning: CommonJS module C:\Users\Femi\AppData\Roaming\npm\node_modules\npm\node_modules\
src\node.js is loading ES Module C:\Users\Femi\AppData\Roaming\npm\node_modules\npm\node_modules\supports-color\in
 using require().
Support for loading ES Module in require() is an experimental feature and might change at any time
(Use `node --trace-warnings ...` to show where the warning was created)

added 12 packages, and audited 53 packages in 2s

10 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

Brcypt for password hashing

```
C:\Users\Femi\OneDrive - Technological University Dublin (1)\Desktop\project\my-app>npm install bcrypt
(node:12184) ExperimentalWarning: CommonJS module C:\Users\Femi\AppData\Roaming\npm\node_modules\npm\node_modules\
src\node.js is loading ES Module C:\Users\Femi\AppData\Roaming\npm\node_modules\npm\node_modules\supports-color\in
 using require().
Support for loading ES Module in require() is an experimental feature and might change at any time
(Use `node --trace-warnings ...` to show where the warning was created)

added 3 packages, and audited 56 packages in 943ms

10 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

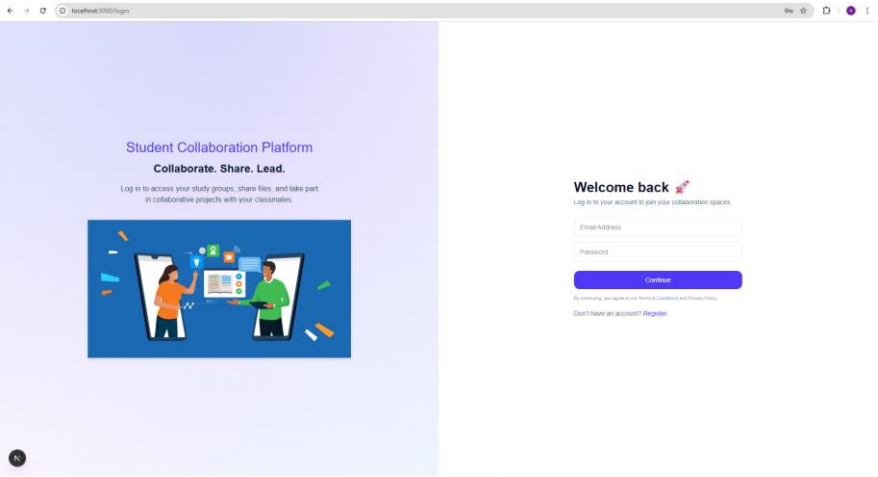
Users field for student collaboration database

Full student record generated in the database after a student register through my web application.

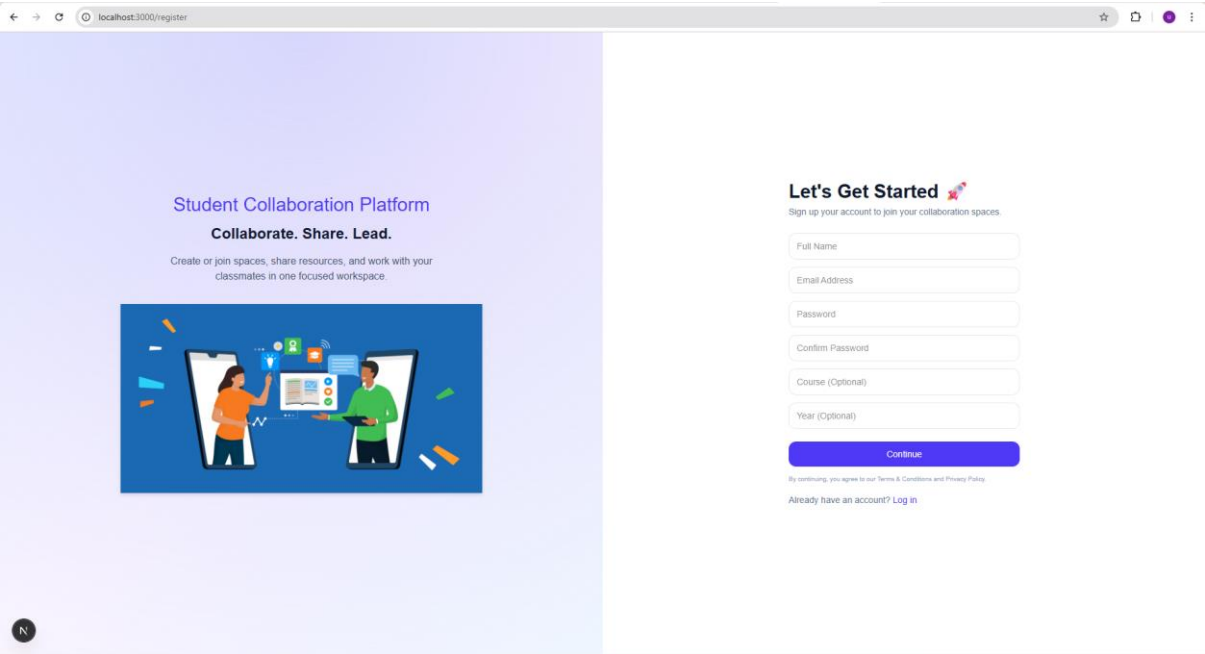
```
_id: ObjectId('6916fe5e8c7f7ac335bb7604')
name : "Full name"
email : "fullname@gmail.com"
password : "$2b$10$z0iPJzqlMyN69M1aGcEdNO48ZMALUbFUUEZtuhtNTDvubGR60ZOgu"
course : ""
year : ""
role : "student"
createdAt : 2025-11-14T10:03:10.942+00:00
```

09/11/2025

Login

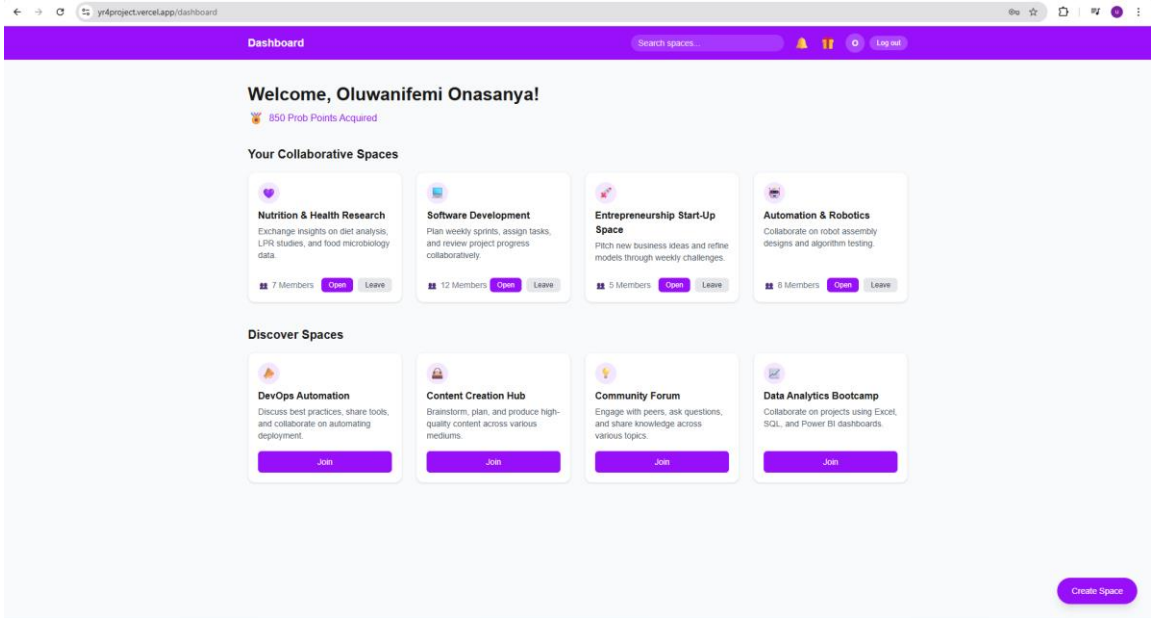


Register



11/11/2025

Dashboard

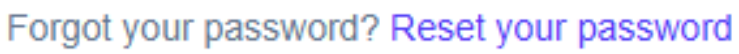


19/11/2025

Installing Node mailer to send emails for resetting passwords

```
C:\Users\Femi\OneDrive - Technological University Dublin (1)\Desktop\project\my-app>npm install nodemailer
(node:14080) ExperimentalWarning: CommonJS module C:\Users\Femi\AppData\Roaming\npm\node_modules\npm\node_modules\debug\
src\node.js is loading ES Module C:\Users\Femi\AppData\Roaming\npm\node_modules\npm\node_modules\supports-color\index.js
 using require().
Support for loading ES Module in require() is an experimental feature and might change at any time
(Use `node --trace-warnings ...` to show where the warning was created)

added 1 package, and audited 58 packages in 726ms

10 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

Resetting password UI for users



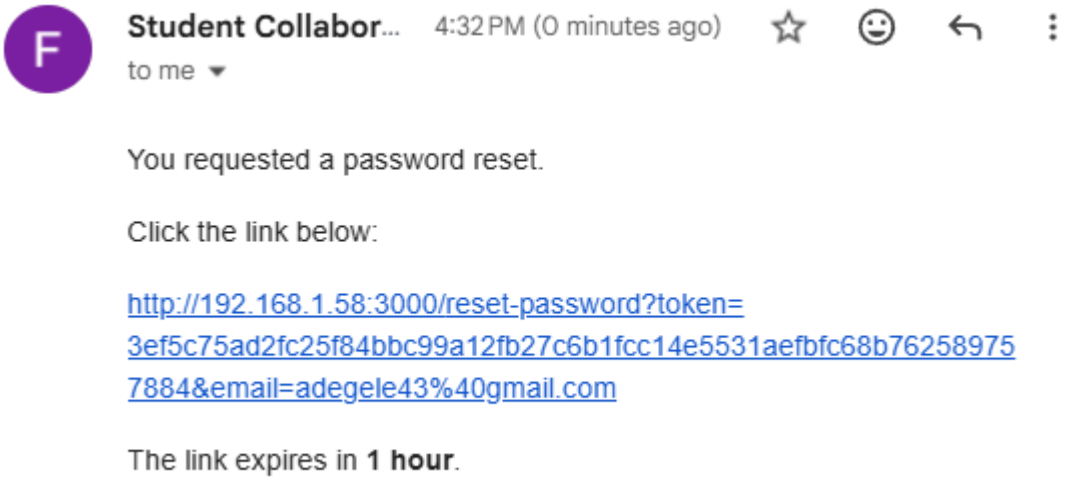
Code that updates new password for the user after resetting

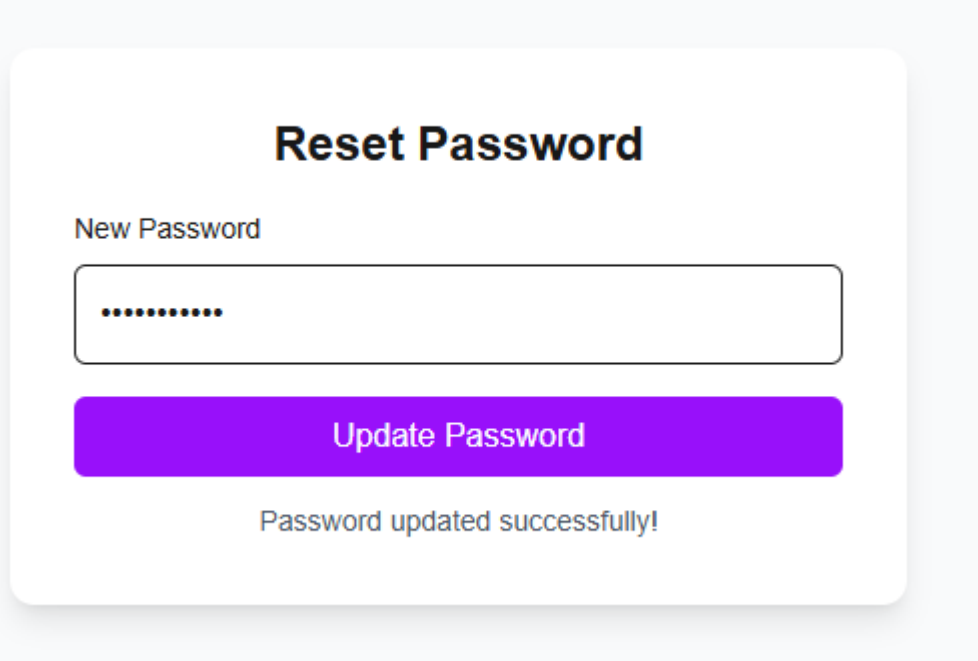
```
// Update password & clear token
await users.updateOne(
  { _id: user._id },
  {
    $set: { password: hashedPassword },
    $unset: { resetTokenHash: "", resetTokenExpiry: "" },
  }
);
```

I implemented a password reset email system using Nodemailer.

This piece of code is responsible for sending the password reset email to users who forgot their password. It uses a tool called Nodemailer which send emails directly to the users Gmail automatically.

```
    // Nodemailer setup
    const transporter = nodemailer.createTransport({
        host: "smtp.gmail.com",
        port: 587,
        secure: false,
        auth: {
            user: process.env.EMAIL_USER,
            pass: process.env.EMAIL_PASS,
        },
    });

    await transporter.sendMail({
        from: `"Student Collaboration" <${process.env.EMAIL_USER}>`,
        to: email,
        subject: "Reset your password",
        html: `
    <p>You requested a password reset.</p>
    <p>Click the link below:</p>
    <p><a href="${resetLink}">${resetLink}</a></p>
    <p>The link expires in <strong>1 hour</strong>.</p>
  `,
    });
```



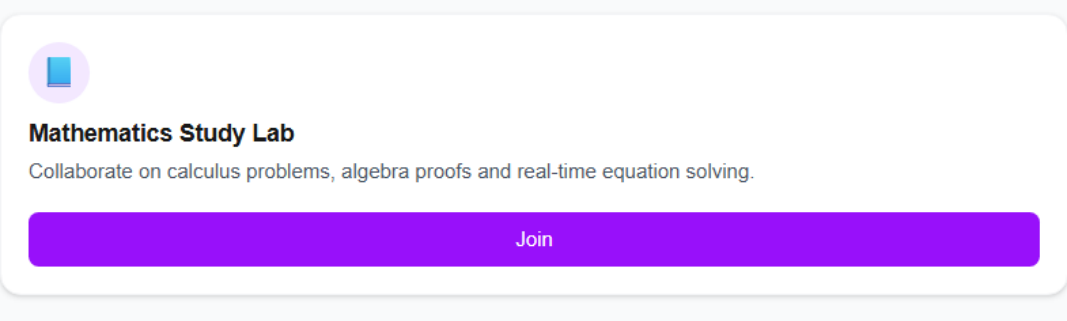


Created a new mongodb collection called Spaces for creating a dynamic space in the dashboard.

Dynamic Space on MongoDb

```
_id: ObjectId('692756253ac966d1154d5248')
title : "Mathematics Study Lab"
desc : "Collaborate on calculus problems, algebra proofs and real-time equatio…"
icon : "📘"
▸ members : Array (empty)
```

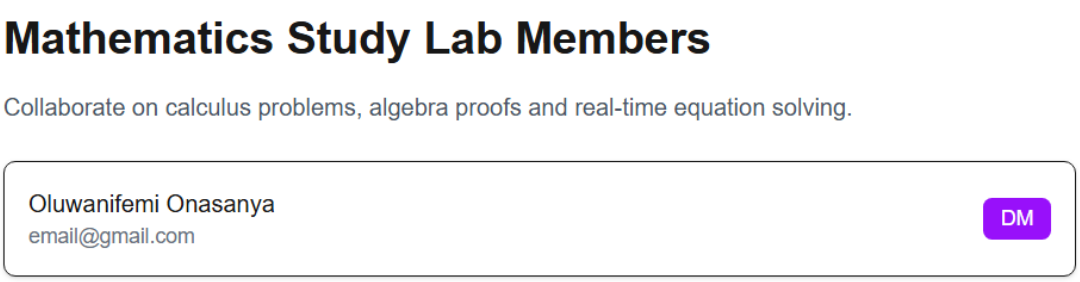
Space shows on dashboard after being fetched from MongoDB



Members added to the space database schema after joining the specific from the dashboard page

```
▸ _id: ObjectId('692756253ac966d1154d5248')
  title : "Mathematics Study Lab"
  desc : "Collaborate on calculus problems, algebra proofs and real-time equatio…"
  icon : "📘"
▾ members : Array (1)
  ▾ 0: Object
      email : "email@gmail.com"
      name : "Oluwanifemi Onasanya"
```

When users join a space, they can see other users in the space.



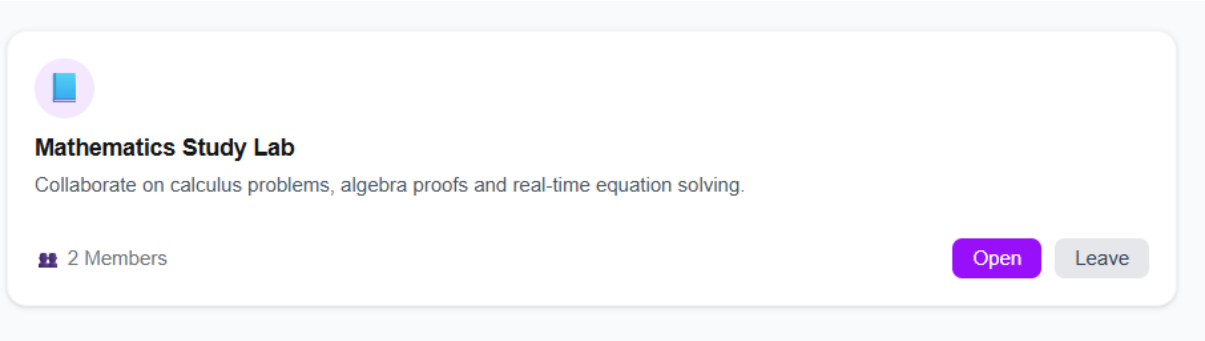
Installing Server version of Socket.io

```
C:\Users\Adege\Desktop\MajorProject\yr4project>npm install socket.io

added 15 packages, and audited 82 packages in 3s

10 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

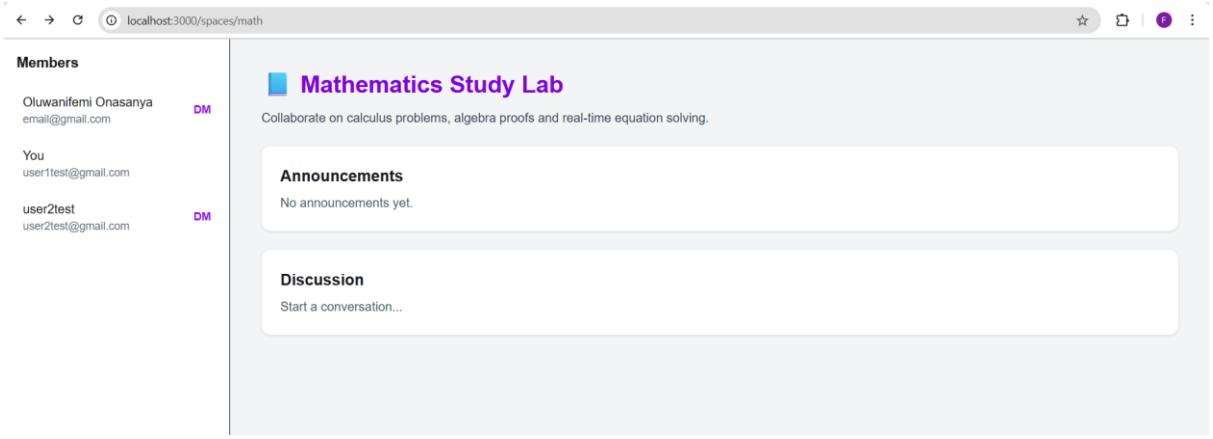
Installing Socket.io Client

```
C:\Users\Adege\Desktop\MajorProject\yr4project>
npm install socket.io-client

added 9 packages, and audited 67 packages in 3s

10 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities

C:\Users\Adege\Desktop\MajorProject\yr4project>
```

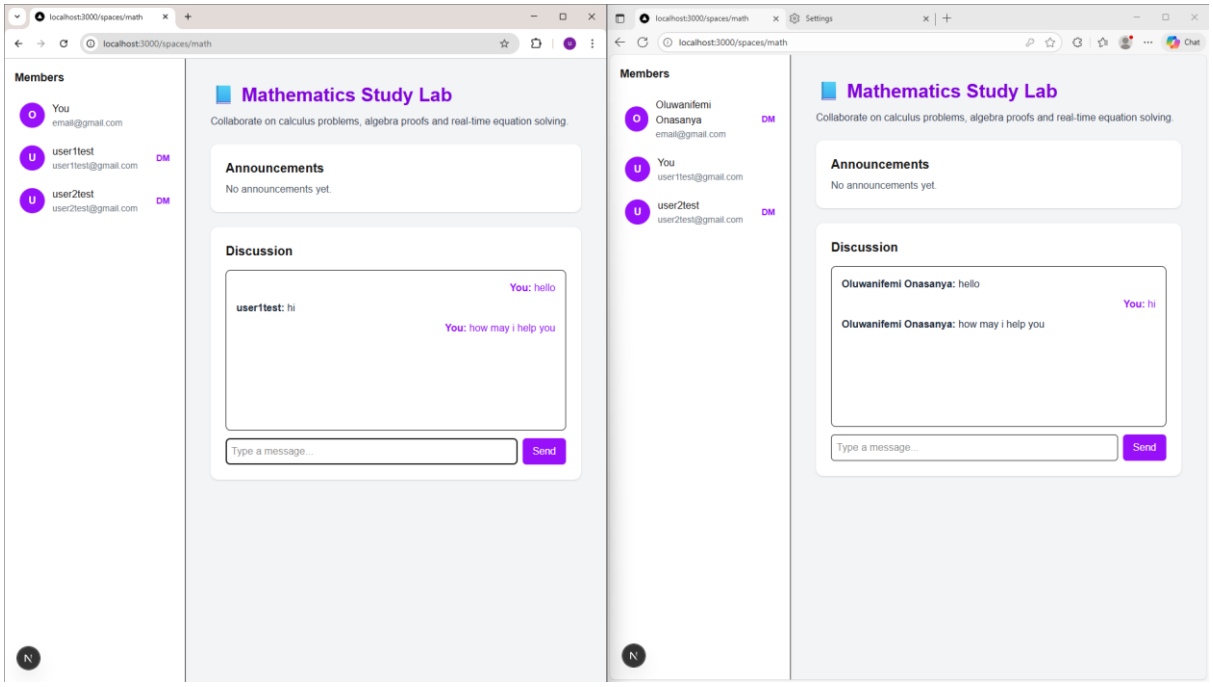
Users can see the Space they joined from discover spaces to their collaborative space



Maths Space redesigned into two column dashboard layout placing members list in a fixed sidebar and improving the main content area.



Socket.io chat messaging functioning in the space.



The server logs show each space message being broadcast with the sender, name and message details

This means the backend is handling message distribution reliably without errors.

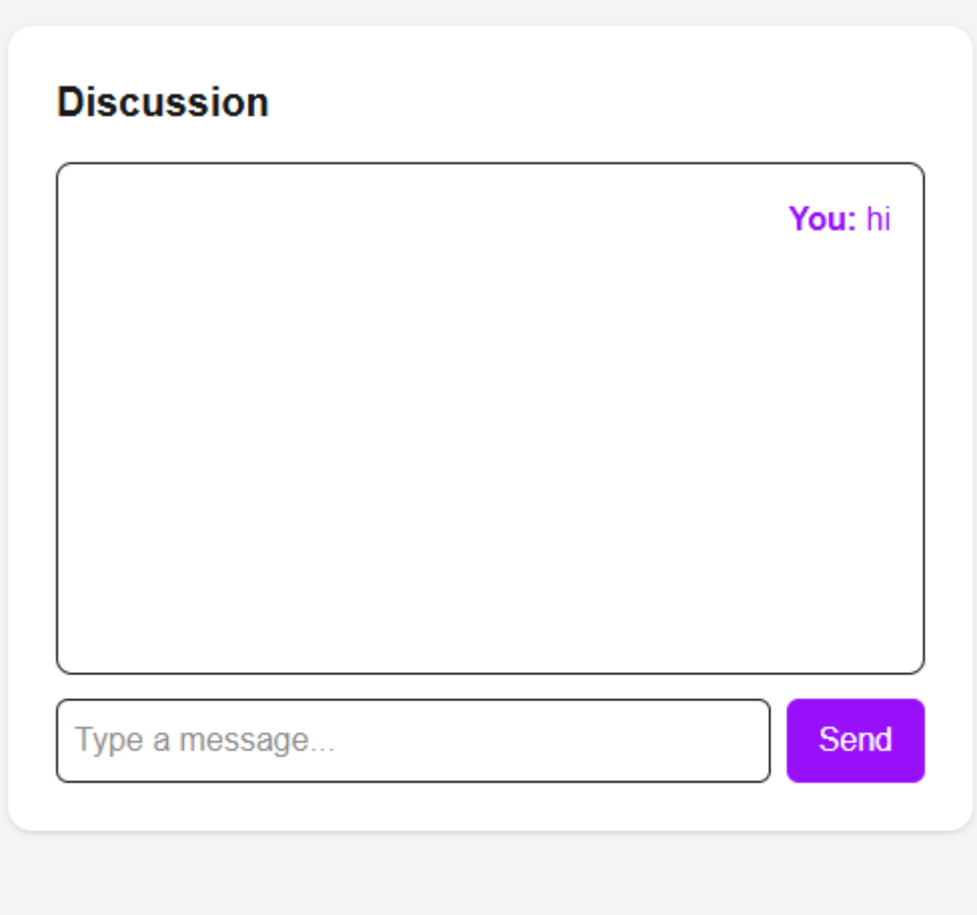
```
Space -> Broadcasting: 692756253ac966d1154d5248
{
  spaceId: '692756253ac966d1154d5248',
  sender: 'email@gmail.com',
  name: 'Oluwanifemi Onasanya',
  message: 'hello '
}
Space -> Broadcasting: 692756253ac966d1154d5248
{
  spaceId: '692756253ac966d1154d5248',
  sender: 'user1test@gmail.com',
  name: 'user1test',
  message: 'hi'
}
Space -> Broadcasting: 692756253ac966d1154d5248
{
  spaceId: '692756253ac966d1154d5248',
  sender: 'email@gmail.com',
  name: 'Oluwanifemi Onasanya',
  message: 'how may i help you'
}
```

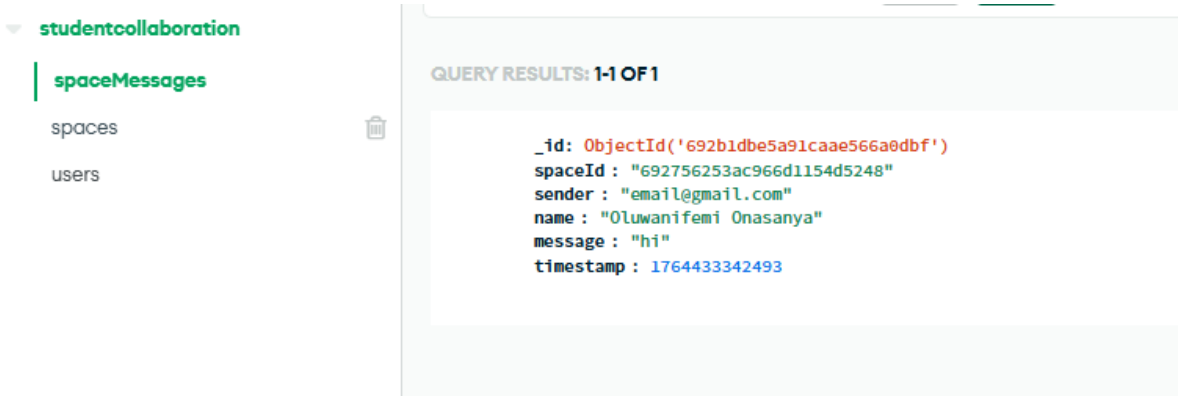
Storing the socket io real time space chat messaging into Mongodb

```
try {
    const { spaceId, sender, name, message, timestamp } = req.body;

    const client = await clientPromise;
    const db = client.db("studentcollaboration");

    const collection = db.collection("spaceMessages");

    await collection.insertOne({
        spaceId,
        sender,
        name,
        message,
        timestamp,
    });
```

When user sends a message it gets stored in the database

For vercel I have to use an external backend as vercel cannot host socket.io





Added a new timestamp formatting function that converts each message raw timestamp into clean, readable time format

This prepares the timestamp so it can be displayed properly in the UI

```
  const formatTime = (ts) => {
  const date = new Date(ts);
  return date.toLocaleTimeString([], { hour: "2-digit", minute: "2-digit" });
};
```

I updated the bubble UI to show the timestamp under each message using the formatTime() helper, this makes the chat more informative by showing when each message was sent.

```
  <p className="text-xs font-semibold mb-1">
    {isMe ? "You" : msg.name}
  </p>
  <p>{msg.message}</p>
  <p className="text-[10px] opacity-70 mt-1 text-right">
    {formatTime(msg.timestamp)}
  </p>
```

This is getDayLabel logic

I added a small function that checks each message's date and shows "Today", "Yesterday" or the full date. This helps to group messages in more natural chat format

```
const formatDay = (ts) => {
  const d = new Date(ts);
  return d.toLocaleDateString([], {
    year: "numeric",
    month: "short",
    day: "numeric",
  });
};

const getDayLabel = (ts) => {
  const messageDate = new Date(ts);
  const today = new Date();

  const diff = today.setHours(0, 0, 0, 0) - messageDate.setHours(0, 0, 0, 0)
  const oneDay = 24 * 60 * 60 * 1000;

  if (diff === 0) return "Today";
  if (diff === oneDay) return "Yesterday";

  return formatDay(ts);
};
```

Before listing the messages, I added a lastDate tracker. This lets the system know when a new starts so it can show a separator like “Today” in the chat

```
{(() => {
  let lastDate = null;

  return messages.map((msg, i) => {
    const dayLabel = getDayLabel(msg.timestamp);
    const isNewDay = dayLabel !== lastDate;
    lastDate = dayLabel;
```

I added a divider inside the chat that appears only when the day changes followed by the actual message bubble and shows at what time

```
return (
  <div key={i}>
    {/* Day Divider */}
    {isNewDay && (
      <div className="text-center my-3">
        <span className="text-xs text-gray-500 bg-gray-200 px-3 py-1 rounded-full">
          {dayLabel}
        </span>
      </div>
    )}

    {/* Message Bubble */}
    <div className={`flex ${isMe ? "justify-end" : "justify-start"}`}>
      <div
        className={`max-w-xs px-4 py-2 rounded-lg shadow-sm ${isMe
          ? "bg-purple-600 text-white rounded-br-none"
          : "bg-gray-200 text-gray-800 rounded-bl-none"
        }`}
      >
        <p className="text-xs font-semibold mb-1">
          {isMe ? "You" : msg.name}
        </p>
        <p>{msg.message}</p>
        <p className="text-[10px] opacity-70 mt-1 text-right">
          {formatTime(msg.timestamp)}
        </p>
      </div>
    </div>
  </div>
);
```

Every message now includes a timestamp

```
const sendMessage = () => {
  if (!chatInput.trim()) return;

  const msg = {
    spaceId: mathSpace._id,
    sender: currentUserEmail,
    name: currentUserName,
    message: chatInput,
    timestamp: Date.now(),
  };

  setMessages((prev) => [...prev, msg]);

  socketRef.current.emit("space-message", msg);

  setChatInput("");
};
```

02/12/2025

I implemented an online feature tracking that users can use to see the activity status of members in the space

Using green for active and red for unactive

This tells the server that the user is now active in the space, triggers the server to update the shared online users list

```
// Identify user as online
socket.emit("user-online", currentUserEmail);
```

The server adds users to an online list when they connect and removes them when they disconnect. It then broadcast the updated list so every client sees who is online instantly.

```
// Online Status
socket.on("user-online", (email) => {
    onlineUsers[email] = socket.id; // mark user as online
    io.emit("online-users", onlineUsers); // update all clients
});
```

```
// Handle disconnect
socket.on("disconnect", () => {
    for (const email in onlineUsers) {
        if (onlineUsers[email] === socket.id) {
            delete onlineUsers[email]; // remove offline user
            break;
        }
    }

    io.emit("online-users", onlineUsers); // this updates all client
});
;
```

User Interface for showing green(online) or red (offline status)

```
        {/* ONLINE STATUS DOT */}
        <span
          className={`absolute -bottom-1 -right-1 w-3 h-3 rounded-full border border-white
 ${(m.email === currentUserEmail || onlineUsers[m.email])
            ? "bg-green-500"
            : "bg-red-500"}`}
        ></span>
      </div>
```

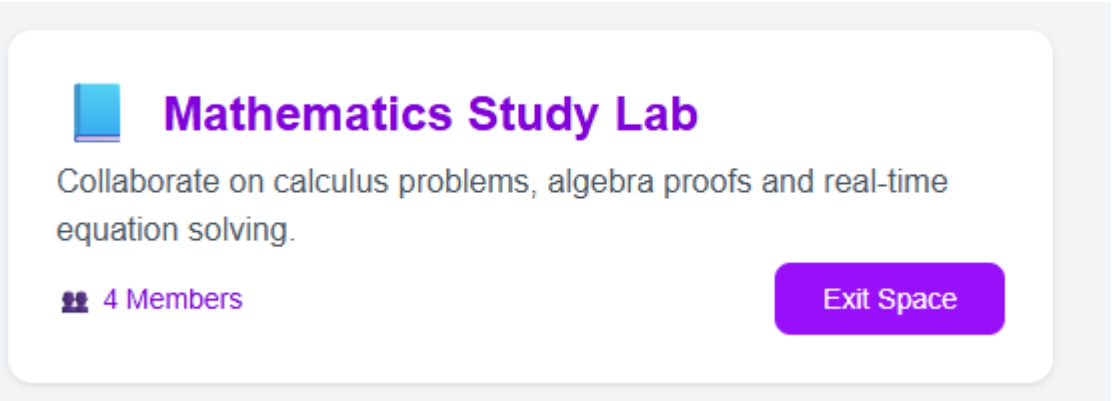
Added exit functionality for the space and users would be redirected to the dashboard page

```
const LeaveSpace = () => {
  window.location.href = "/dashboard";
};
```

Code to create the Exit Space User Interface

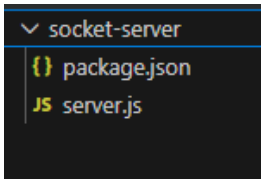
```
    <button
  onClick={LeaveSpace}
  className="bg-purple-600 hover:bg-purple-700 text-white px-6 py-2 rounded-lg"
>
  Exit Space
```

UI Displayed for users to exit the space



06/12/2025

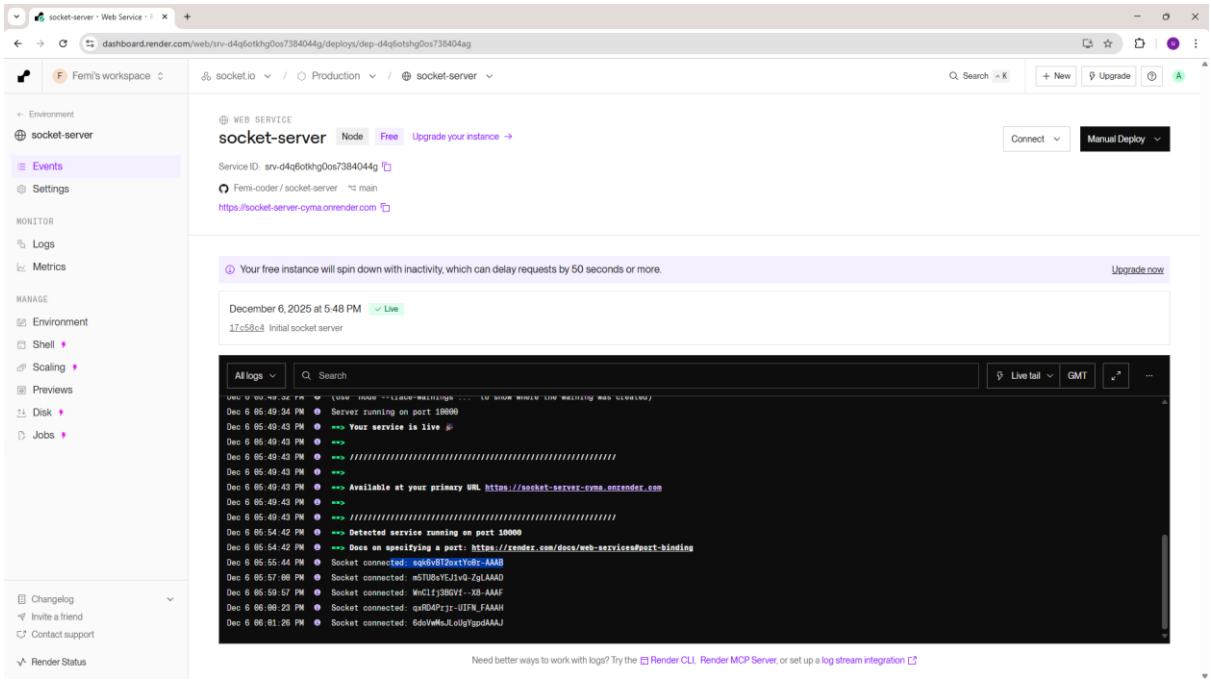
I created a dedicated socket-server directory inside my project, containing a package.json and server.js file. Since Vercel does not support long-running WebSocketservers, I needed to host my Socket.io backend externally.



In the frontend, I updated my Socket.io client initialization to use the Render URL instead of localhost.

```
if (!socketRef.current) {
  socketRef.current = io("https://socket-server-cyma.onrender.com", {
  transports: ["websocket"],
  });
}
```

Today, I reviewed the deployment logs for my Socket.io server hosted on Render.
The logs confirmed that the service successfully deployed and is running correctly on the assigned port.



07/12/2025

This code saves each DM message in the database so messages remain after refresh

This fixed my issue initially where dm messages disappeared on refresh. Now it stores every message permanently

```
// Save to db
await fetch("/api/saveDM", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(msg)
});
```

This fetch loads message history when the DM page opens.

This would retrieve earlier messages from MongoDB so the chat shows full history like WhatsApp

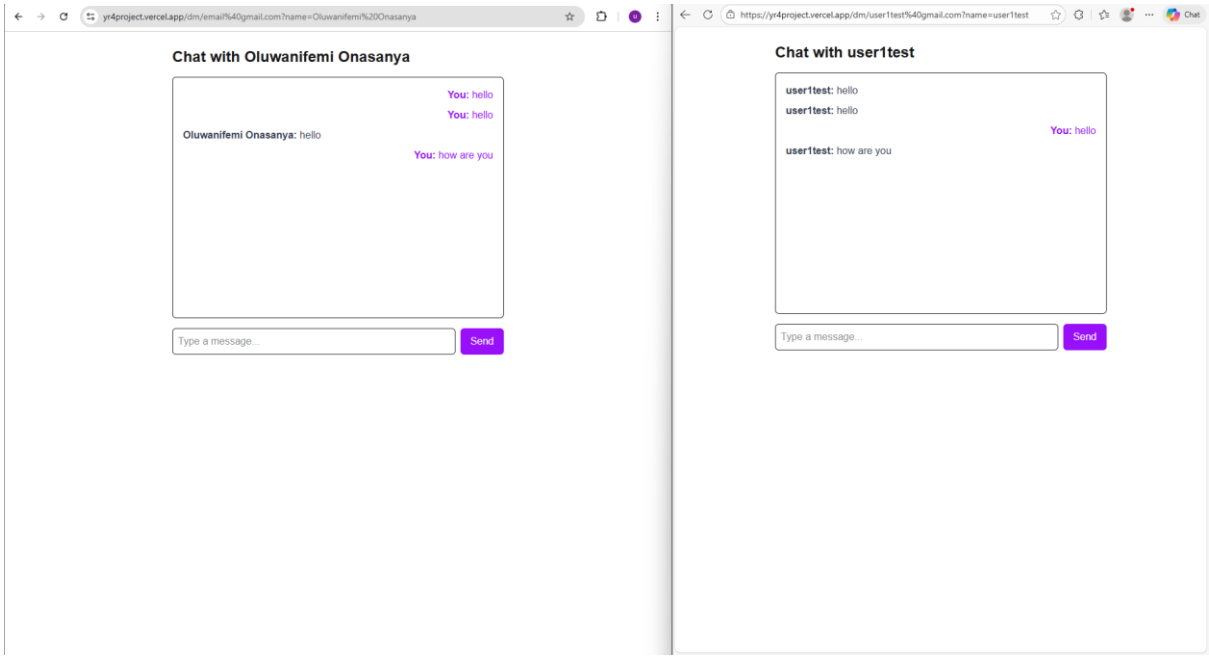
```
fetch(`/api/getDM?roomId=${room}`)
    .then(res => res.json())
    .then(data => {
        setMessages(data.messages || []);
```

This is the Render socket server broadcasting DM messages

This shows how real time communication works, they are instantly delivered to the correct room

```
socket.on("send-message", (data) => {
    io.to(data.roomId).emit("receive-message", data);
});
```

This screenshot shows the completed Direct Messaging feature, where two different user accounts communicate in real time using the deployed Socket.IO server on Render. Messages appear instantly on both sides without refreshing and each message is also saved to MongoDB, allowing the full chat history to reload when the page is refreshed or reopened.

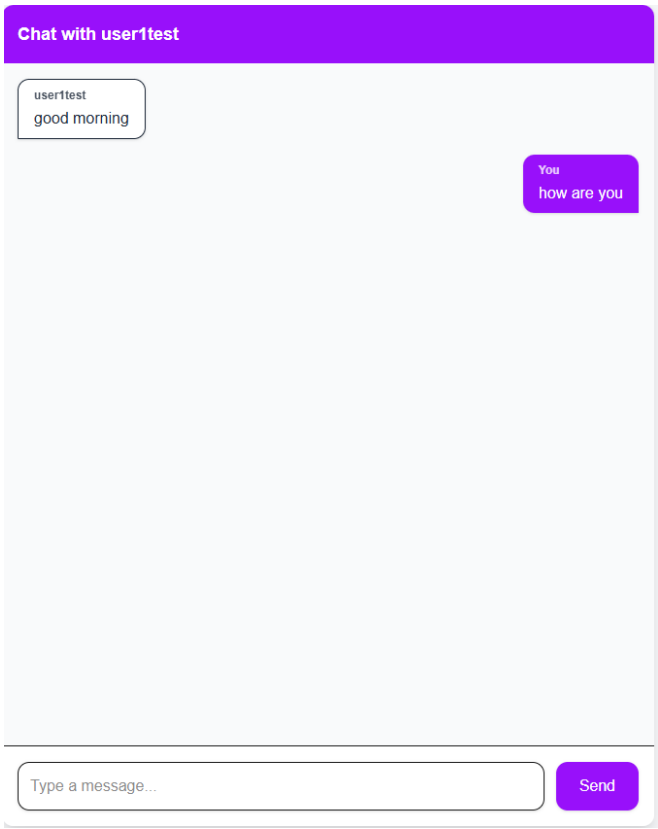


08/12/2025

I configured the app to use a local socket server during development and the Render server in production. This allowed my localhost and the Vercel deployment to connect to the same real-time system correctly.

```
if (!socketRef.current) {

    const SOCKET_URL =
        process.env.NODE_ENV === "development"
            ? "http://localhost:4000"
            : "https://socket-server-cyma.onrender.com";

    socketRef.current = io(SOCKET_URL);
}
```

Also enhanced the Direct Messaging UI



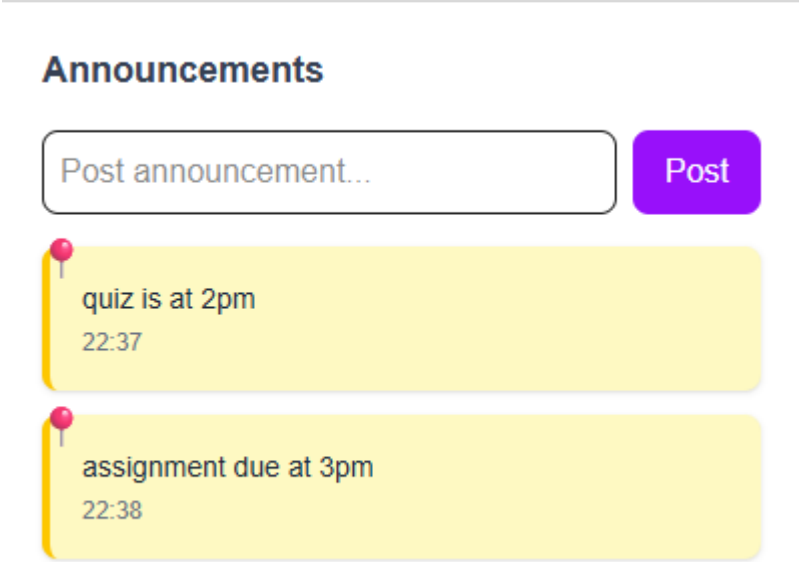
I added this announcement feature so members of the study space can quickly post important updates or reminders. Each announcement is stored in state and displayed instantly in the sidebar, improving communication and keeping users informed in real time.

```
<button
  onClick={() => {
    if (!annInput.trim()) return;

    setAnnouncements((prev) => [
      ...prev,
      {
        text: annInput,
        time: new Date().toLocaleTimeString([], {
          hour: "2-digit",
          minute: "2-digit",
        }),
      },
    ]);

    setAnnInput("");
  }}
  className="bg-purple-600 hover:bg-purple-700 text
>
  Post
</button>
```

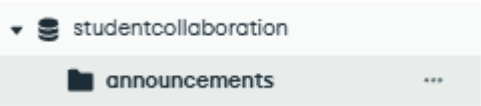
Using a sticky-note style makes announcements stand out visually, helping users instantly recognize important updates. The bright colour, pin icon and card layout improve readability and create a clear separation from the chat messages which ensures that important information doesn't get lost in the conversation.

```
{/* Notes */}
<div className="space-y-3 flex flex-col items-start">
  {announcements.length === 0 && (
    <p className="text-sm text-gray-500">No announcements yet.</p>
  )}

  {announcements.map((a, i) => (
    <div
      key={i}
      className="relative bg-yellow-100 border-l-4 border-yellow-400
          p-4 rounded-lg shadow-sm w-full"
    >
      <span className="absolute -left-2 -top-2 text-xl">📌</span>

      <p className="text-sm font-medium text-gray-800">{a.text}</p>
      <p className="text-xs mt-1 text-gray-500">{a.time}</p>
```

This is how the announcements feature appears in the user interface, users can type an update and click Post and it displays immediately as a styled sticky note with a pin icon and timestamp.



In order to add real time announcements in the space I created a mongodb collection “announcements”



When a user sends an announcement it shows in the database collection announcement


```
_id: ObjectId('69381edb9b3e75486f6e57b3')
spaceId : "test123"
sender : "test@test.com"
text : "Manual test announcement"
timestamp : 1765285595863

_id: ObjectId('69381fb59b3e75486f6e57b4')
spaceId : "test123"
sender : "test@test.com"
text : "Manual test announcement"
timestamp : 1765285812900

_id: ObjectId('6938201a9b3e75486f6e57b5')
spaceId : "test123"
sender : "test@test.com"
text : "Manual test announcement"
timestamp : 1765285913979

_id: ObjectId('69382d726a40e11584cd377a')
spaceId : "692756253ac966d1154d5248"
sender : "adegele43@gmail.com"
text : "hi"
timestamp : 1765289330570
```

Semester 2

09/02/2026 - 15/02/2026

Changing the hardcoded/ Static space from Semester 1 into Dynamic spaces for scalability.

Static space from Semester 1

```
const mySpacesStatic = [
  {
    title: "Nutrition & Health Research",
    desc: "Exchange insights on diet analysis, LPR studies, and food microbiology data.",
    members: 7,
    icon: "💜",
  },
  {
    title: "Software Development",
    desc: "Plan weekly sprints, assign tasks, and review project progress collaboratively.",
    members: 12,
    icon: "💻",
  },
  {
    title: "Entrepreneurship Start-Up Space",
    desc: "Pitch new business ideas and refine models through weekly challenges.",
    members: 5,
    icon: "🚀",
  },
  {
    title: "Automation & Robotics",
    desc: "Collaborate on robot assembly designs and algorithm testing.",
    members: 8,
    icon: "🤖",
  },
];

const discoverSpaces = [
  {
    title: "DevOps Automation",
    desc: "Discuss best practices, share tools, and collaborate on automating deployment.",
    icon: "🚀",
  },
  {
    title: "Health & Wellbeing",
    desc: "Support mental health, physical fitness, nutrition, lifestyle habits, and balance."
    members: 14,
    icon: "💚",
  },
  {
    title: "Social Care Practice",
    desc: "Discuss care work, community support, safeguarding, and social care studies.",
    members: 12,
    icon: "❤️",
  },
  {
    title: "Data Analytics Bootcamp",
    desc: "Collaborate on projects using Excel, SQL, and Power BI dashboards.",
    icon: "📊",
  },
];
```

Dynamic space

```
const [spaces, setSpaces] = useState([]);
```

```
useEffect(() => {
  fetch("/api/getSpaces")
    .then(res => res.json())
    .then(data => setSpaces(data.spaces || []));
}, []);
```

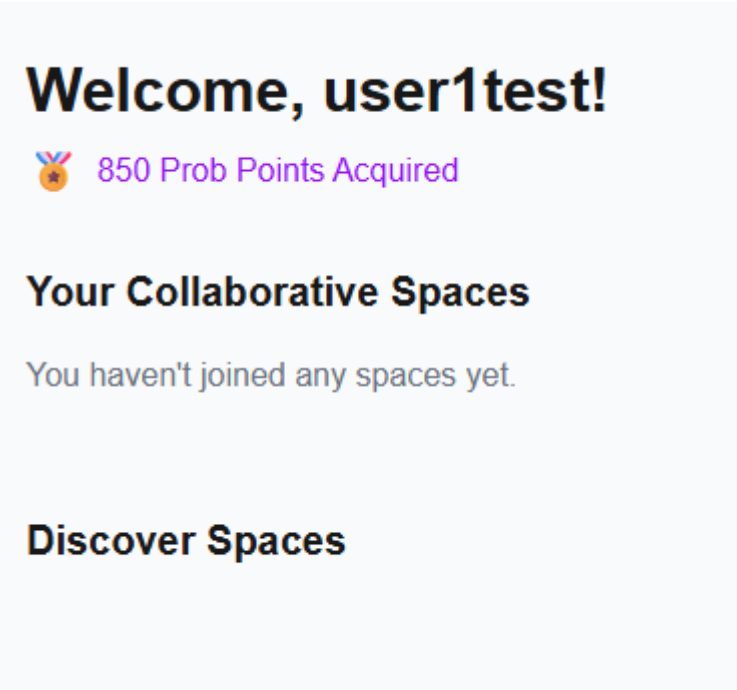
This is used to load all spaces dynamically

Spaces have unique IDs, Spaces store their own discussions, Spaces store their own announcements, Spaces have different members.

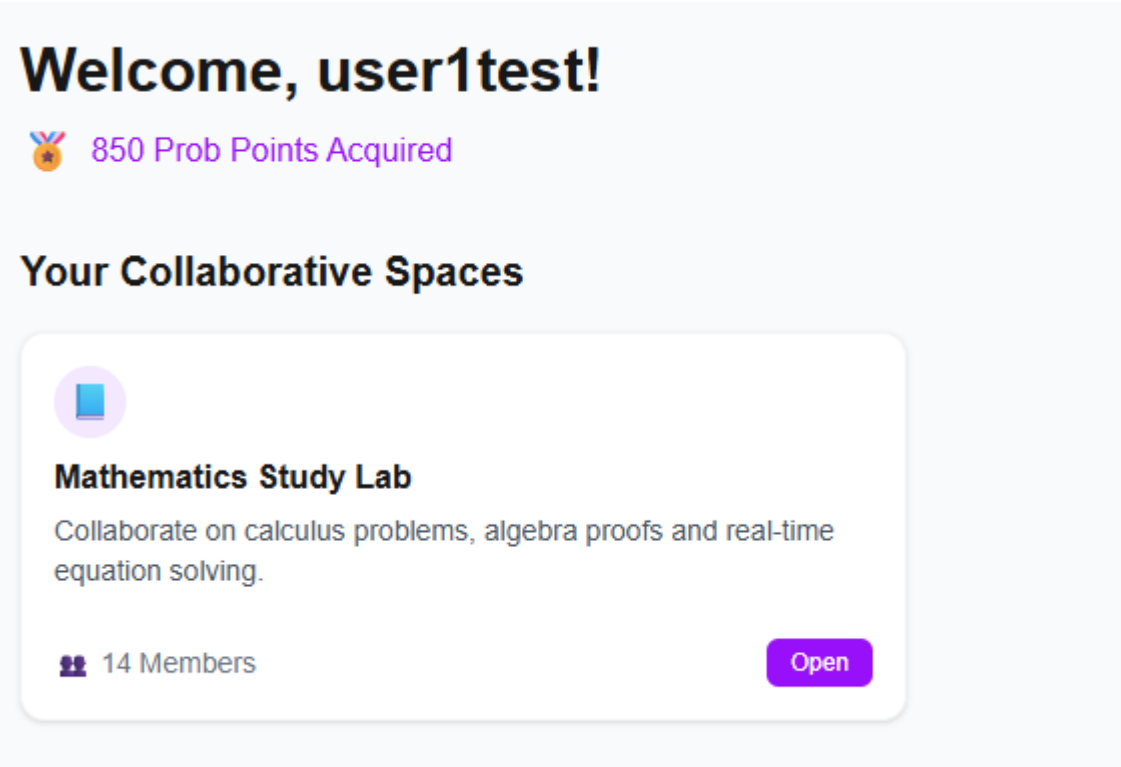
I have to create a dynamic space page

```
spaces
JS [id].js        U
```

Showing the user hasnt joined any space because of switching to dynamic spaces



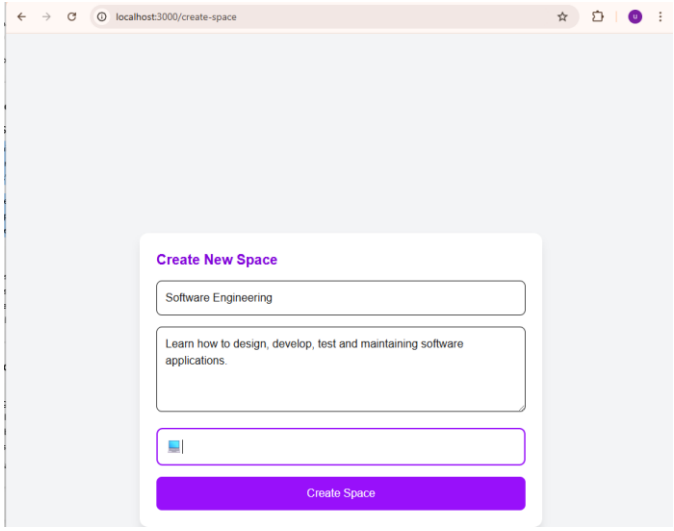
Appears as soon as the user joins it



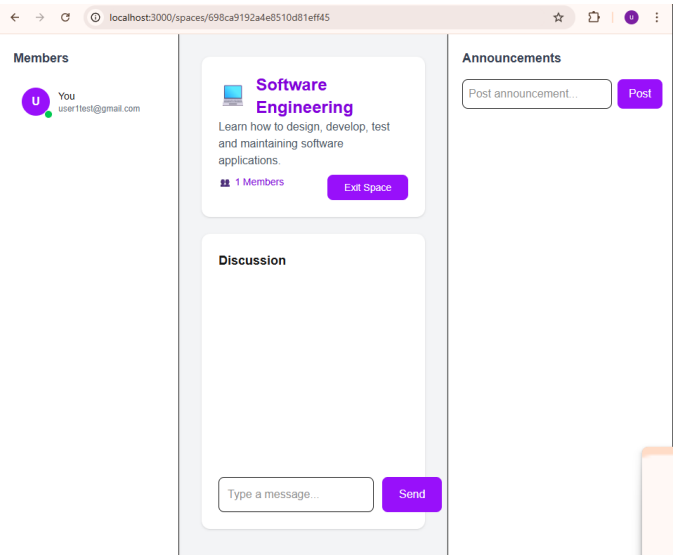
Implementing discovering Spaces

I implemented the dynamic Discover Spaces feature. When a user creates a new collaborative space, it is automatically stored in MongoDB and becomes visible to other users in the Discover Spaces section of their dashboard.

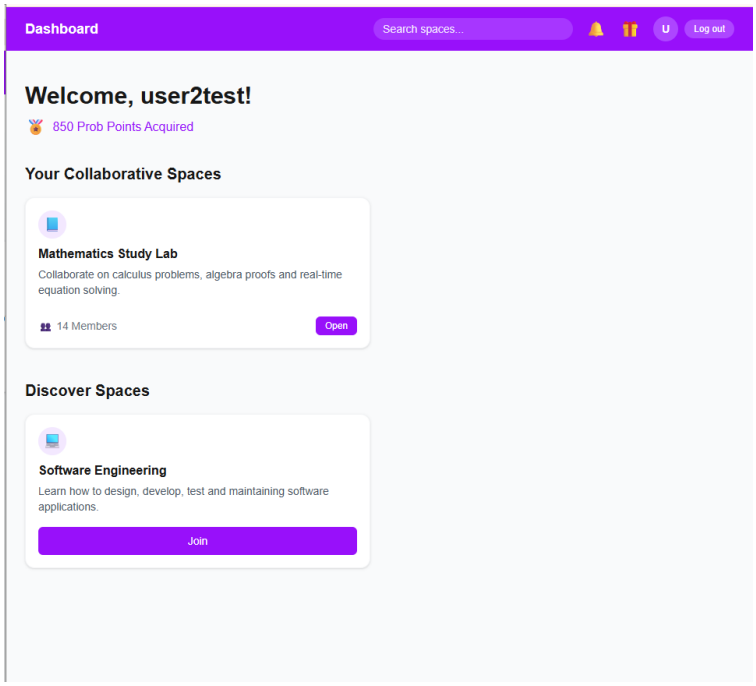
The system filters spaces based on membership. If a user is not already listed in the space's members array, the space appears in their Discover section with an option to join. Once the user joins, the space moves to their Your Collaborative Spaces section.



Space is created by the user and automatically loads and dynamically allocates a fresh UI for that space



Shows in user2test in discover spaces as it was created by user1test and user2test isnt a member yet

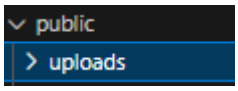


File Sharing in Spaces

I installed the multer library to have access to uploading files.

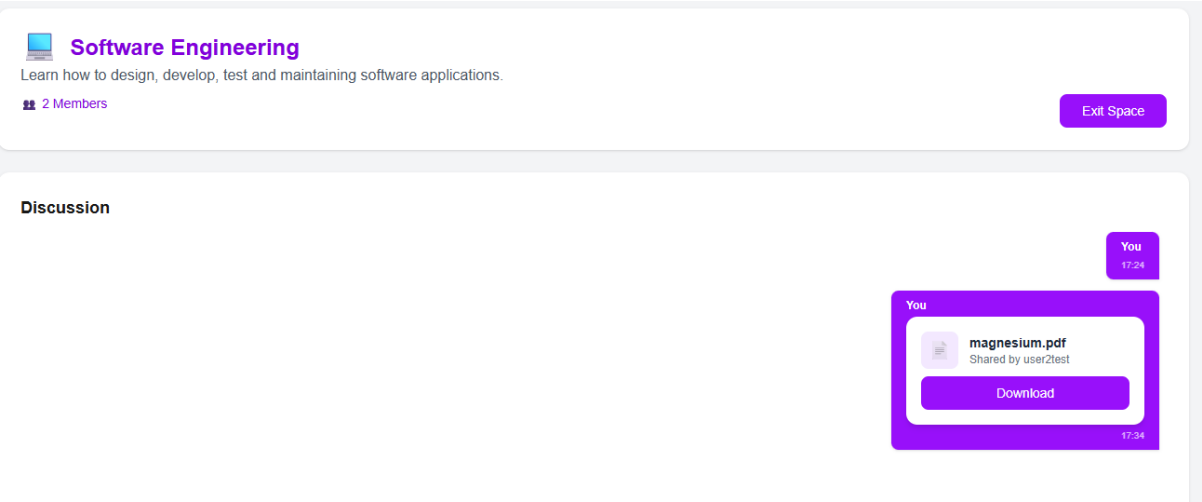
```
C:\Users\Femi\OneDrive - Technological University Dublin (1)\Desktop\project\my-app>npm install multer
```

I created an upload folder in my project root inside the public folder



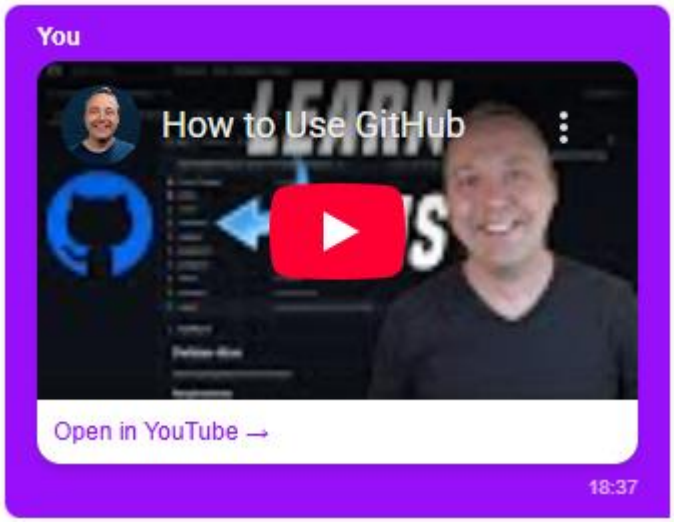
I created an upload Api to allow for file sharing in the backend

I added an Attach button UI to Discussion Input in the space



Users can attach a file now and share in the collaboration space, ready to be downloadable by other users.

Users can share embedded YouTube videos in the discussion section which becomes a playable version of the YouTube video and has a clickable text open in YouTube option which redirects the user to the video in the original YouTube site.



16/02/2026 - 22/02/2026

Point System implementation

I first needed to update my registration Api so registered users are assigned a default 0 points at the beginning.

```
// Gamification fields
points: 0,
reactionsReceived: 0,
reactionsGiven: 0,
```

I created a reactToMessage API to implement 3 types of reactions, reaction removal, reaction change support.

JS reactToMessage.js U

```
import clientPromise from "../../lib/mongodb";
import { ObjectId } from "mongodb";

const reactionPoints = {
    like: 5,
    clap: 10,
    fire: 15,
};
```

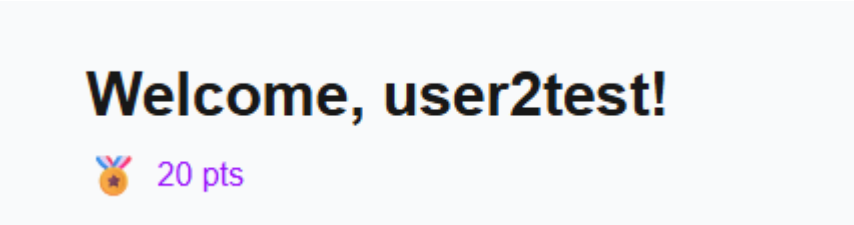
I added a message reaction UI under each message

```
{/* REACTIONS*/}
<div className="flex gap-3 mt-2 text-sm">
    {["like", "clap", "fire"].map((type) => {
        const count =
            msg.reactions?.filter((r) => r.type === type).length || 0;

        const emojiMap = {
            like: "👍",
            clap: "👏",
            fire: "🔥",
        };

        return (
            <button
                key={type}
                onClick={() => handleReaction(msg._id, type)}
                className="flex items-center gap-1 hover:scale-110 transition"
            >
                <span>{emojiMap[type]}</span>
                <span>{count}</span>
            </button>
```

Points displayed in the database with a user reaction to the message (5 points)

```
_id: ObjectId('690dfd2161e04d9af4b97c1b')
name : "Femi Onasanya"
email : "adegele43@gmail.com"
password : "$2b$10$uWz2DcRVUJ7zIpwnmrjnbOsQpICKwaqWKZ663Fh2LxcNkG3tM5Odq"
course : "Computing"
year : "4"
role : "student"
createdAt : 2025-11-07T14:07:29.553+00:00
points : 5
```

Displaying of points is accessible in dashboard, not hardcoded anymore.



Leaderboard System for Global leaderboard Card

I created a leaderboard system to display the top 5 users with the highest points in the dashboard.

I will first create a getLeaderboardAPI

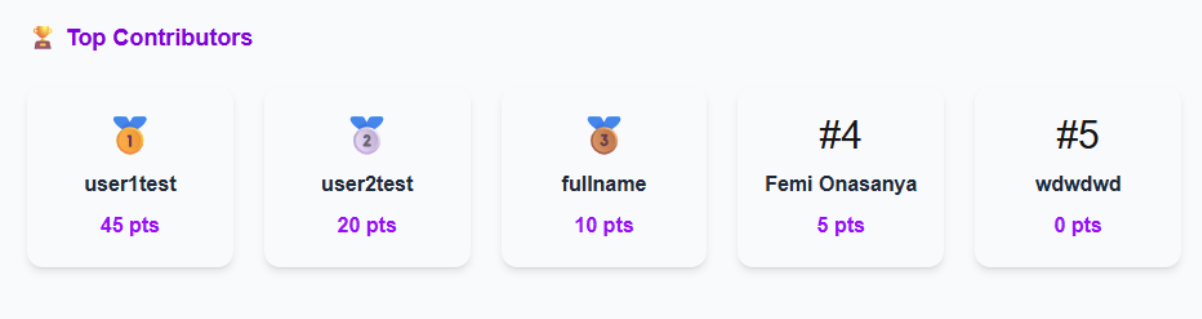
JS getLeaderboard.js U

```
const topUsers = await db
  .collection("users")
  .find({})
  .sort({ points: -1 })
  .limit(5)
  .project({ name: 1, points: 1 })
  .toArray();

return res.status(200).json({ leaderboard: topUsers });
```

Added a state in dashbord.js

```
const [leaderboard, setLeaderboard] = useState([]);
```

Added a Leaderboard Card UI that displays in the dashboard for every user to view



23/02/2026 - 01/03/2026

Making Leaderboards Clickable to display in depth details such as points gained from each group and spaces they belong too.

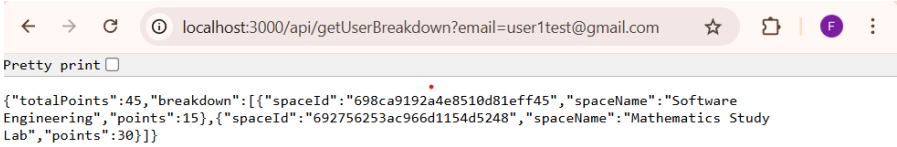
Created a getUserBreakdown.js Api file to Loop through messages

For each message it will Check reactions, Sum reaction points amd Group by spaceId

And would Return totalPoints and breakdown array

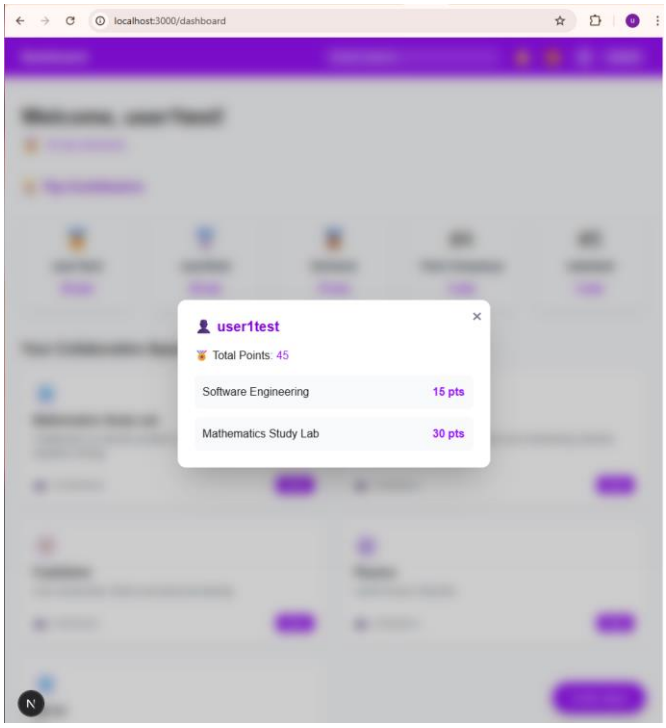
```
const breakdownArray = Object.keys(spaceBreakdown).map((spaceId) => ({
    spaceId,
    spaceName: spaceMap[spaceId] || "Unknown Space",
    points: spaceBreakdown[spaceId],
}));
```

Successfully Testing the getUserBreakdown in the localhost browser



{"totalPoints":45,"breakdown":[{"spaceId":"698ca9192a4e8510d81eff45","spaceName":"Software Engineering","points":15},{"spaceId":"692756253ac966d1154d5248","spaceName":"Mathematics Study Lab","points":30}]}

Enhanced clickable leaderboard UI that displays points earned per space



Notification System Implementation

I added a notification state in the dashboard

```
const [hasNotification, setHasNotification] = useState(false);
```

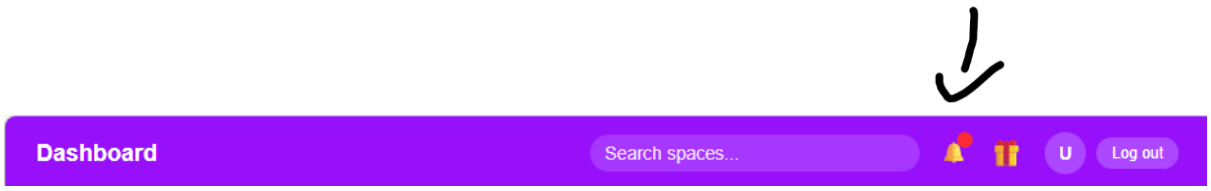
I connected the socket in the dashboard

```
useEffect(() => {
  if (!user?.email || spaces.length === 0) return;

  const SOCKET_URL =
    process.env.NODE_ENV === "development"
      ? "http://localhost:4000"
      : "https://socket-server-cyma.onrender.com";

  const socket = io(SOCKET_URL, { transports: ["websocket"] });

  // Join ALL spaces user belongs to
  const mySpaces = spaces.filter((space) =>
    space.members.some((m) => m.email === user.email)
  );

  mySpaces.forEach((space) => {
    socket.emit("join-space", space._id);
  });

  // Listen for space messages
  socket.on("space-message", (data) => {
    if (data.sender !== user.email) {
      setHasNotification(true);
    }
  });

  // Listen for DMs
  socket.on("receive-message", (data) => {
    if (data.sender !== user.email) {
      setHasNotification(true);
    }
  });
```

Updated my Bell icon to display red when a notification occurs

```
{hasNotification && (
  <span className="absolute -top-1 -right-1 w-3 h-3 bg-red-500 rounded-full"></span>
)}
```

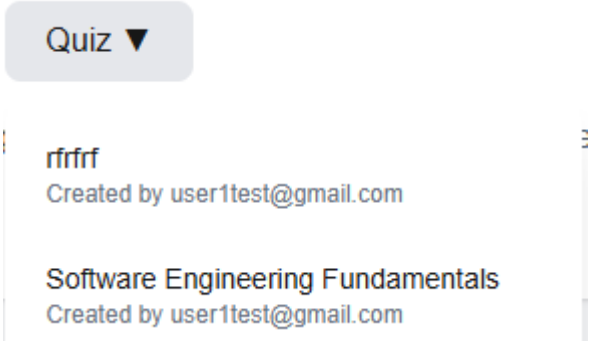
Bell Notification result in the UI



Quiz Feature Implementation

I implemented dynamic Quiz Feature per space.

Designed UI to allow users to view available quizzes via dropdown and create new quizzes inside a specific space



I built CreateQuizForm component for adding multiple questions, setting correct answers and Saving quizzes to MongoDB

```
function CreateQuizForm({ spaceId, userEmail, onCreated }) {
    const [title, setTitle] = useState("");
    const [question, setQuestion] = useState("");
    const [options, setOptions] = useState(["", "", "", ""]);
    const [answer, setAnswer] = useState(0);
    const [questions, setQuestions] = useState([]);
```

I implemented createQuiz API route to validate inputs and store quizzes inside spaces collection

```
JS createQuiz.js                U
```

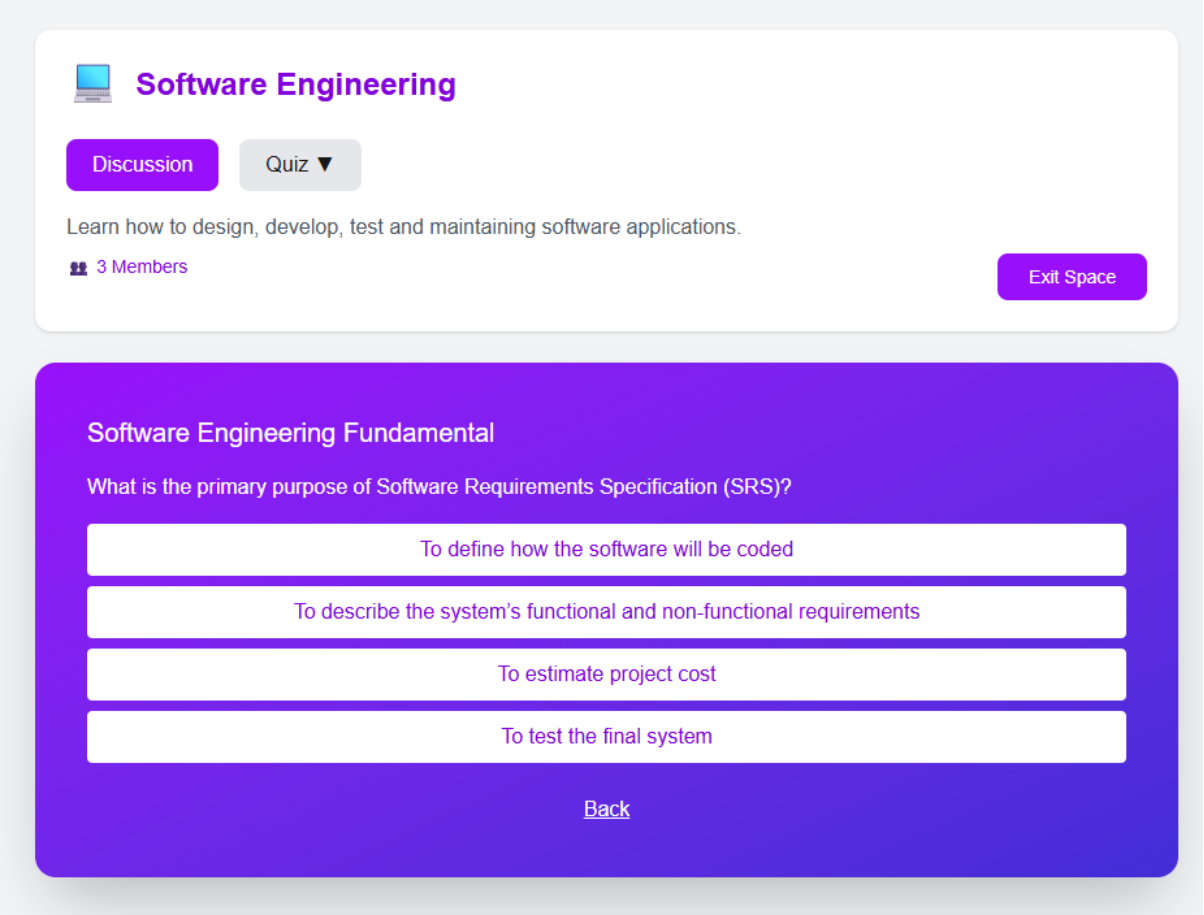
```
const result = await db.collection("spaces").updateOne(
  { _id: new ObjectId(spaceId) },
  {
    $push: { quizzes: newQuiz },
  }
);
```

```
const newQuiz = {
  _id: new ObjectId(),
  title,
  createdBy,
  createdAt: Date.now(),
  questions,
};
```

I developed Quiz Player Component to display questions sequentially, track score dynamically and handle correct answer logic using functional state updates

```
function QuizPlayer({ quiz, onBack }) {
    const [index, setIndex] = useState(0);
    const [score, setScore] = useState(0);

    const current = quiz.questions[index];
```

Quiz Player UI



02/03/2026 - 08/03/2026

Enhancing the Quiz Feature by highlighting the correct answer in each question

```
const [selected, setSelected] = useState(null);
const [showResult, setShowResult] = useState(false);
```

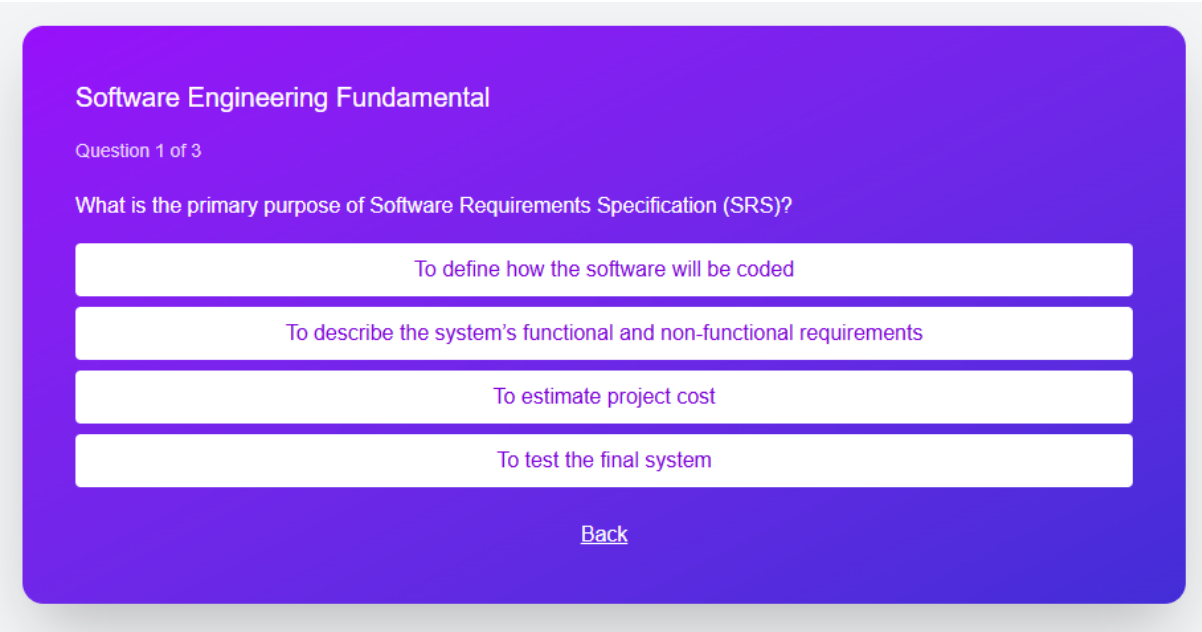
```
if (showResult) {
    if (i === current.answer) {
        buttonColor = "bg-green-500 text-white";
    } else if (i === selected) {
        buttonColor = "bg-red-500 text-white";
    }
}
```

If the chosen answer is correct green button colour would appear

If the chosen answer is wrong red button colour would appear

Adding a Quiz Counter to help users track quiz progress

```
<p className="text-sm text-purple-200 mb-5">
    Question {index + 1} of {quiz.questions.length}
</p>
```



Adding points gained from quiz to the overall quiz total to display in the dashboard

Create an API to Add Quiz Points

```
JS addQuizPoints.js    U
```

```
//add points to users
await db.collection("users").updateOne(
    { email },
    { $inc: { points: points } }
);
```

To allow the user to only have one attempt for a quiz

I created a state for attempts

```
const [completedQuizzes, setCompletedQuizzes] = useState([])
```

Created an API to return Quiz Attempts

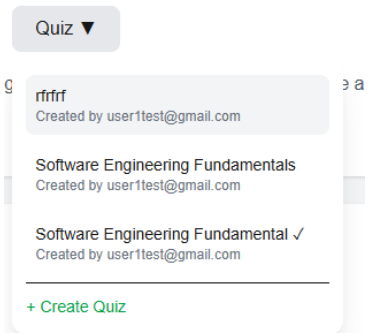
JS getQuizAttempts.js U

```
const attempts = await db
    .collection("quizAttempts")
    .find({ userEmail: email })
    .toArray();
```

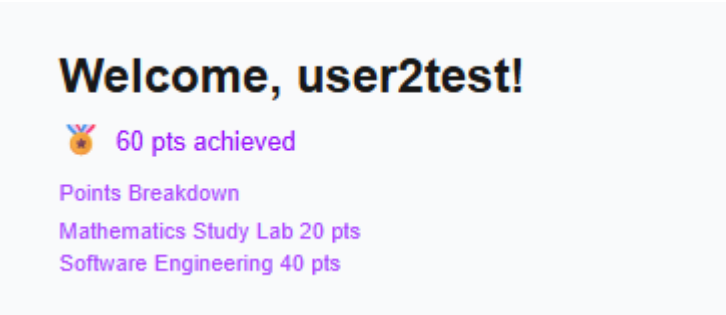
Quiz Attempts collection in the database that tracks the users quiz attempt

```
_id: ObjectId('69a979c1d50793ca931169a9')
quizId : "69a5a3a70f7d6636c10743be"
userEmail : "user1test@gmail.com"
score : 30
completedAt : 2026-03-05T12:40:33.720+00:00

_id: ObjectId('69a97e79d50793ca931169aa')
quizId : "69a5a3a70f7d6636c10743be"
userEmail : "user2test@gmail.com"
score : 20
completedAt : 2026-03-05T13:00:41.855+00:00
```

Implemented a visual indicator within the Space interface that displays a tick beside quiz titles to signify quizzes that have already been completed by the logged in user.



I Implemented a personalised points breakdown feature within the dashboard. This functionality retrieves the logged in user's contribution data from the backend and displays the points earned across each collaborative space directly beneath the total points indicator. This ensures that users can easily monitor their individual contributions regardless of their ranking on the leaderboard.



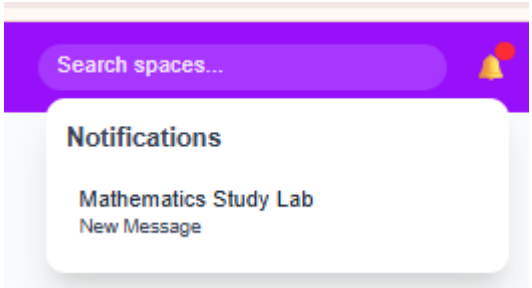

```
{userBreakdown?.breakdown?.length > 0 && (
  <div className="mt-2 text-purple-500 text-sm">

    <p className="font-medium mb-1">
    Points Breakdown
    </p>

    {userBreakdown.breakdown.map((item, index) => (
      <div key={index}>
        {item.spaceName} {item.points} pts
      </div>
    ))}

  </div>
)}
```

I Updated the backend logic to ensure quiz points are correctly associated with their corresponding collaborative spaces. This was achieved by storing the spaceId alongside quiz attempts, allowing the getUserBreakdown API to aggregate both reaction points and quiz scores per space.

09/03/2026 - 15/03/2026

I Implemented an improved notification system in the dashboard to provide clearer alerts when new activity occurs in the collaborative spaces.

I added a notification dropdown menu accessible from the bell icon in the navigation bar.



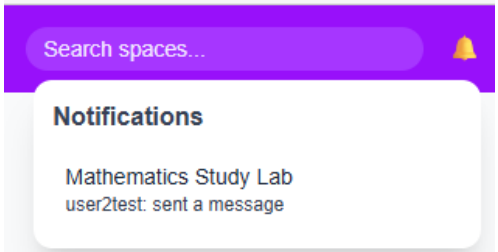
I configured the system to display the name of the collaboration space where the message originated from which allows users to identify the source of the notification

```
const msg = {
    spaceId: id,
    spaceName: space.title,
    sender: currentUserEmail,
    name: currentUserName,
    type,
    content: chatInput.trim(),
    timestamp: Date.now(),
};
```

Added spaceName:space.title to retrieve the space name for every message being sent by the user.

I also want to make the senders name display in the notification dropdown to inform the other users in the space who the sender's name is

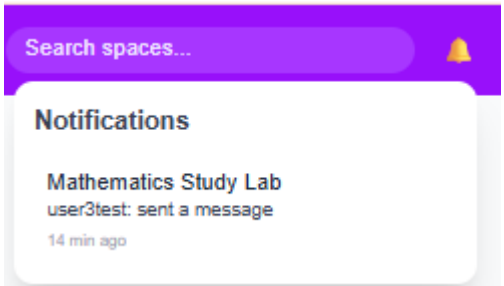
```
message: `${data.name}: sent a message`,
```



Integrated notification timestamps in the notification system to make the user keep track of all the notifications.

```
const formatTimeAgo = (timestamp) => {
  const seconds = Math.floor((Date.now() - timestamp) / 1000);
```

```
<p className="text-gray-400 text-[10px] mt-1">
  {formatTimeAgo(n.timestamp)}
```



Shows the timestamp of the message in the Dashboard UI (14 mins ago)

Starting to Implement MediaRecorderAPI for audio recording feature accessible in the discussion page which would allow users to record audio, stop recording, audio gets uploaded and message appears in the chat with the audio player

I added new state variables for recoding state media recorder instance and captured audio data

Npm install formidable (package necessary for audio recording in Next.js)

```
C:\Users\Femi\OneDrive - Technological University Dublin (1)\Desktop\project\my-app>npm install formidable
(node:5492) ExperimentalWarning: CommonJS module C:\Users\Femi\AppData\Roaming\npm\node_modules\npm\node_modules\debug\s
rc\node.js is loading ES Module C:\Users\Femi\AppData\Roaming\npm\node_modules\npm\node_modules\supports-color\index.js
using require().
Support for loading ES Module in require() is an experimental feature and might change at any time
(Use `node --trace-warnings ...` to show where the warning was created)

added 7 packages, and audited 106 packages in 2s

14 packages are looking for funding
  run `npm fund` for details
```

```
const [recording, setRecording] = useState(false);
const [mediaRecorder, setMediaRecorder] = useState(null);
const [audioChunks, setAudioChunks] = useState([]);
```

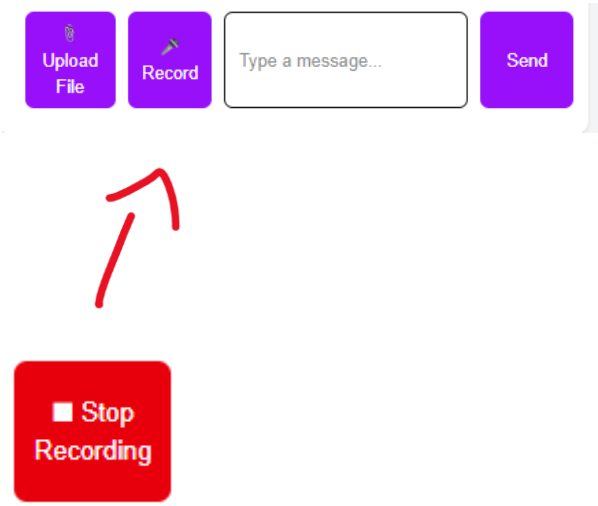
```
const startRecording = async () => {

    console.log("Recording started");
    try {

        const stream = await navigator.mediaDevices.getUserMedia({ audio: true });

        const recorder = new MediaRecorder(stream);

        const chunks = [];

        recorder.ondataavailable = (event) => {
            chunks.push(event.data);
        };
```

This allows users to capture audio using their device microphone and interact with the collaborative space in a more dynamic way.

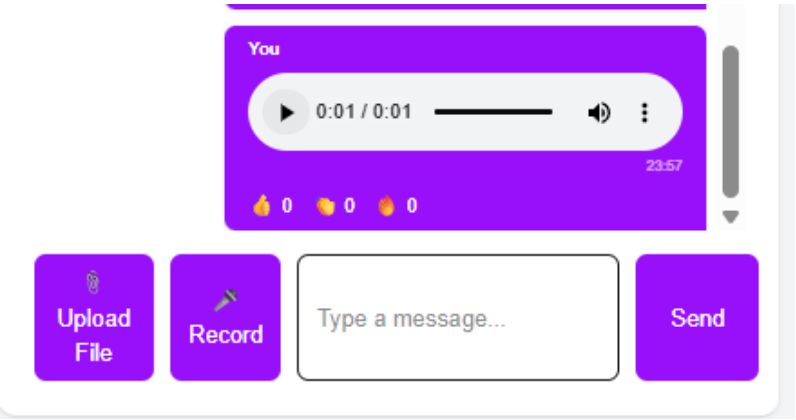
Developed an API endpoint /api/uploadAudio using formidable to handle audio file uploads.

JS uploadAudio.js U

Developed a recording control mechanism allowing users to start and stop recording using a microphone button. The interface dynamically updates to display a stop recording option while audio capture is active.



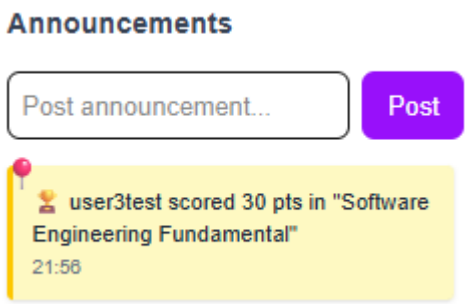
Audio Recording displayed in the Discussion section accessible for other users to listen too.



16/03/2026 - 22/03/2026

I Enhanced the Announcement system in the space to automatically broadcast quiz outcomes which enhances engagement and awareness within collaborative spaces.

```
text: `🏆 ${currentUserName} scored ${finalScore} pts in "${quiz.title}"`,
```




```
_id: ObjectId('69bb1f7a5291ef4c72ba0011')
spaceId : "698ca9192a4e8510d81eff45"
sender : "user3test@gmail.com"
senderName : "user3test"
text : "🏆 user3test scored 30 pts in "Software Engineering Fundamental""
level : "quiz"
timestamp : 1773870970383
```

I started on implementing the Speech to text feature API which listens to the user's microphone and convers speech into text in real time

Added new state components

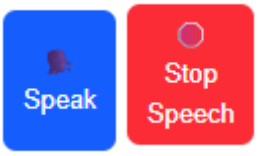
```
const [listening, setListening] = useState(false);
const recognitionRef = useRef(null);
```

Key Object Web Speech API

```
const SpeechRecognition =
    window.SpeechRecognition || window.webkitSpeechRecognition;
```

Speech to Text UI Code

```
<button
    onClick={listening ? stopSpeechToText : startSpeechToText}
    className={`px-3 py-2 rounded-lg text-white ${listening ? "bg-red-500" : "bg-blue-600"
        }`}
>
    {listening ? "🔴 Stop Speech" : "🗣 Speak"}
</button>
```



Implementing Auto scroll in discussion section for better User experience

```
const bottomRef = useRef(null);
const messagesContainerRef = useRef(null);
```

A reference (bottomRef) was introduced at the end of the message container, and the scrollIntoView() method was used to navigate to the most recent message upon component render and message updates.

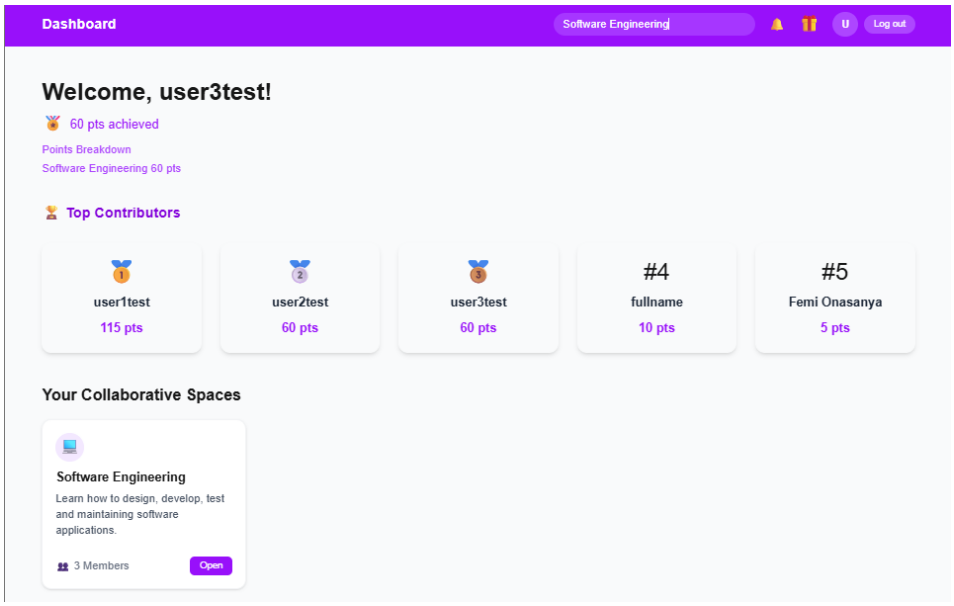
```
useEffect(() => {
    if (messages.length > 0) {
        bottomRef.current?.scrollIntoView({ behavior: "auto" });
    }
}, [messages.length]);
```

A search feature was implemented within the dashboard to enhance navigation and usability. A state variable (search) was introduced to capture user input from the search bar. The list of available spaces was then dynamically filtered using string matching on the space title and description.

The filtered results were subsequently divided into “My Spaces” and “Discover Spaces” based on user membership. This approach ensured real-time updates without requiring additional backend queries, improving performance and user experience.

```
const [search, setSearch] = useState("");
```

```
const filteredSpaces = spaces.filter((space) =>
  space.title.toLowerCase().includes(search.toLowerCase()) ||
  space.desc.toLowerCase().includes(search.toLowerCase))
);

const mySpaces = filteredSpaces.filter((space) =>
  space.members.some(
    (m) => m.email.trim().toLowerCase() === user.email.trim().toLowerCase()
  )
);

const discoverSpaces = filteredSpaces.filter(
  (space) =>
    !space.members.some(
      (m) => m.email.trim().toLowerCase() === user.email.trim().toLowerCase()
    )
);
```



Rotating Leadership Implementation

I started with adding Leader field to createSpaceApi to allocate first member becomes leader and timestamp is saved

```
  //Rotating Leadership
  leader: creatorEmail,
  leaderIndex: 0,
  lastRotation: Date.now(),
});
```

I created a Rotation Leader Api to keep track of the 1-week duration the group leader for rotating

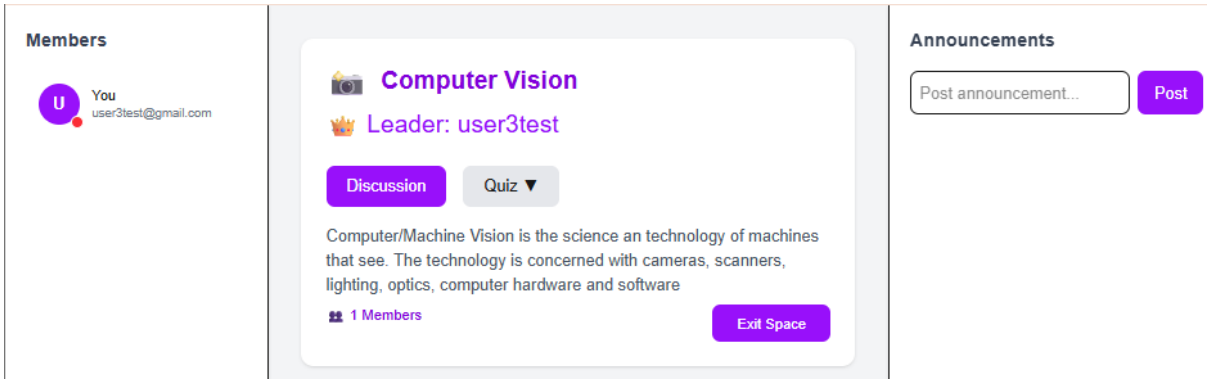
JS rotateLeader.js U

```
    // Rotates only after 7 days passes
    if (Date.now() - space.lastRotation < ONE_WEEK) continue;

    // Calculates the next Leader index
    const nextIndex =
        (space.leaderIndex + 1) % space.members.length;
    const newLeader = space.members[nextIndex].email;

    await db.collection("spaces").updateOne(
        { _id: new ObjectId(space._id) },
        {
            $set: {
                leader: newLeader,
                leaderIndex: nextIndex,
                lastRotation: Date.now(),
            },
        }
    );
}
```

This ensures fair distribution of responsibility and supports structured collaboration within each space.

I displayed the Leader in the Space UI

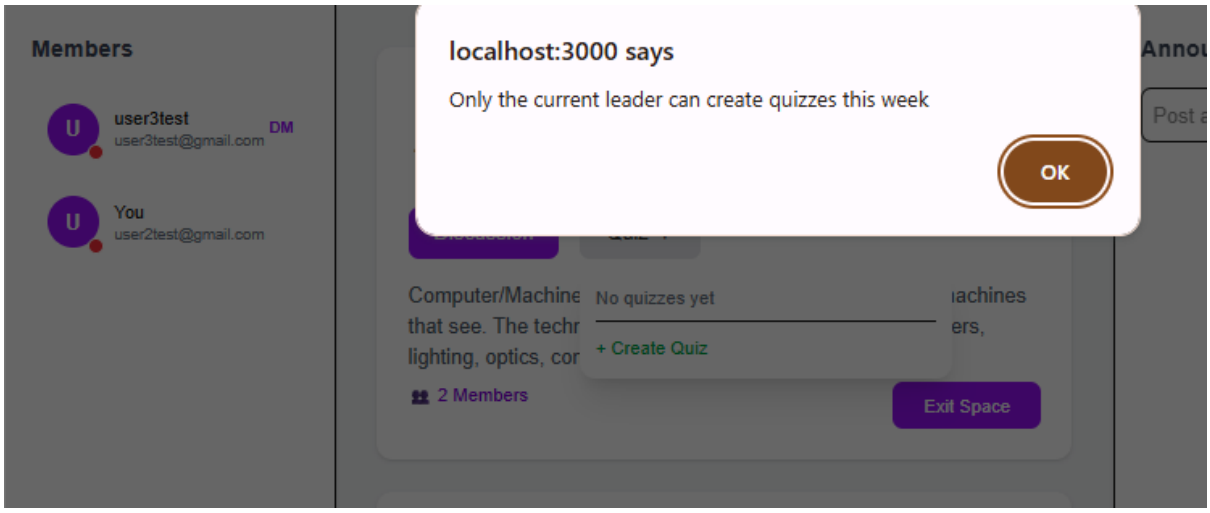
```
<p className="text-2xl text-purple-600 mt-2">
    👑 Leader: {
        space.members.find(m => m.email === space.leader)?.name || "None"
    }
</p>
```



This ensures fair distribution of responsibility and supports structured collaboration within each space.

I restricted access on the creating quiz only to current leader

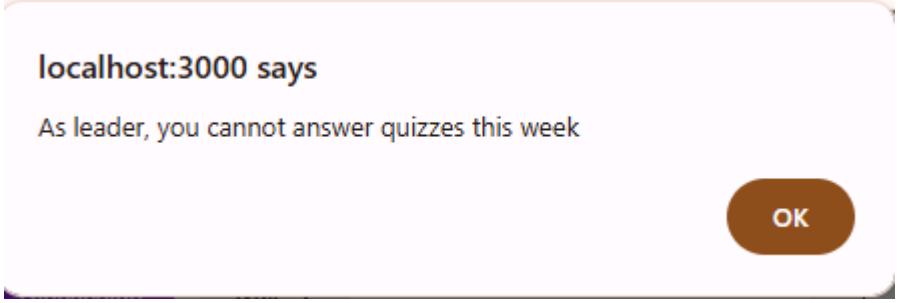
```
<div className="border-t pt-2">
    <button
        onClick={() => {
            if (!isLeader) {
                alert("Only the current leader can create quizzes this week");
                return;
            }
```



Leader cannot access the quiz for ensuring transparency

```
if (isLeader) {
    alert("As leader, you cannot answer quizzes this week");
    return;
}
```

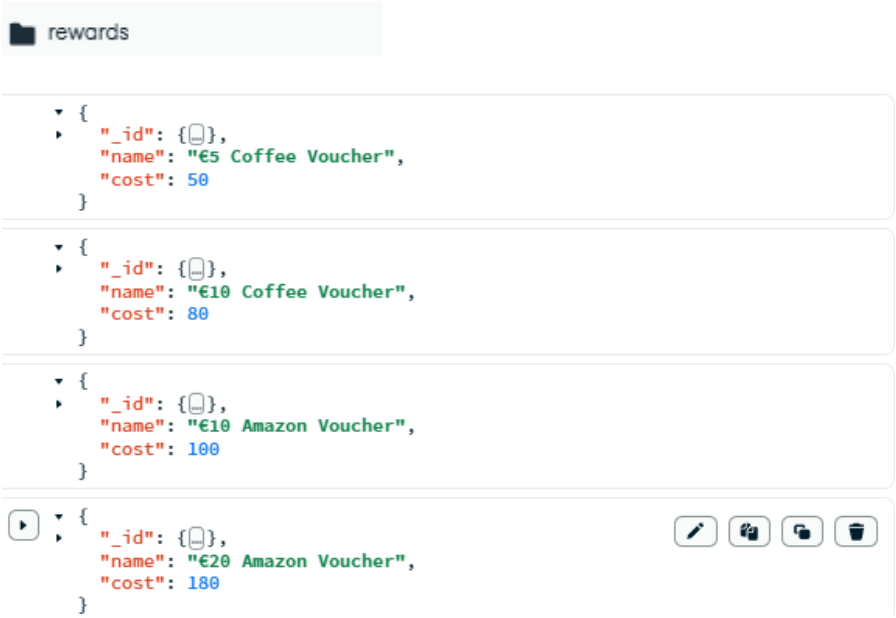
UI message



23/03/2025 - 29/03/2025

Integration of Reward System

Created a reward collection in my MongoDB Database



Created a Redeem Reward API check if the user has enough points and rejects if there aren't sufficient points.

JS redeemReward.js U

JS getRewards.js U

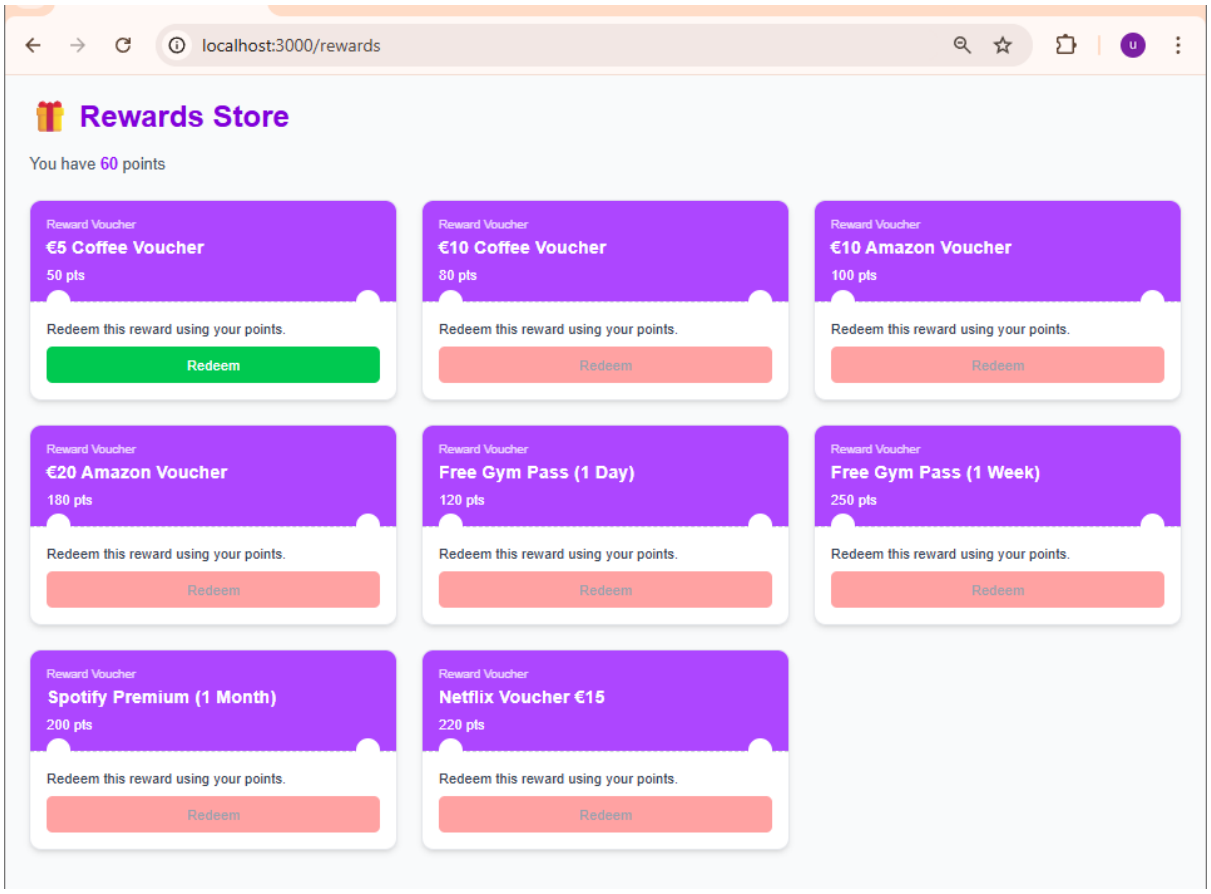
This redirects users to the rewards page from the dashboard page

```
<button
  onClick={() => router.push("/rewards")}
  className="text-xl"
  title="Rewards"
>
  🎁
</button>
```

Created a rewards page to display rewards to users to redeem their rewards using their respective points

JS rewards.js U

Rewards Store UI displaying Various Rewards available showing redeemable ones and unredeemable ones based on points allocation.



Integration of Redeemed Vouchers Section

Created getRewardHistoryAPI to fetch user reward history

JS getRewardHistory.js U

```
const history = await db
    .collection("rewardHistory")
    .find({ email })
    .sort({ timestamp: -1 })
    .toArray();

res.status(200).json({ history });
```

Added state on rewards page to retrieve reward history for the users

```
const [history, setHistory] = useState([]);
```

A real-time message search feature was implemented, allowing users to efficiently locate conversations within collaborative discussions.

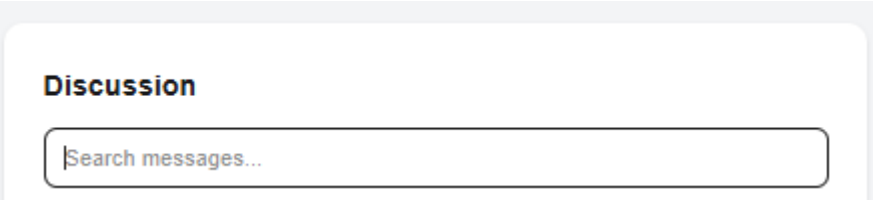
Added search state

```
const [searchChat, setSearchChat] = useState("");
```

Search Input

```
<input
    type="text"
    placeholder="Search messages..."
    value={searchChat}
    onChange={(e) => setSearchChat(e.target.value)}
    className="w-full mb-3 px-3 py-2 border rounded-lg text-sm outline-none"
/>
```

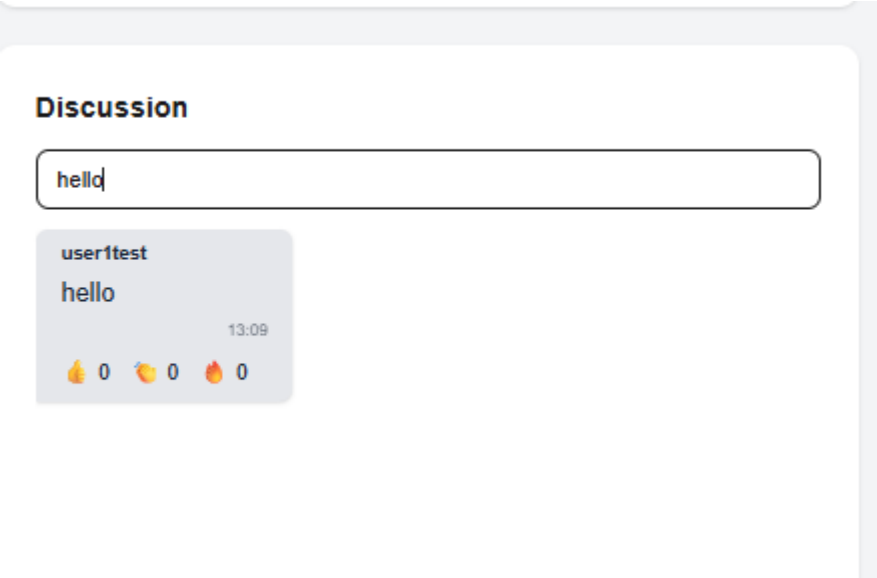
Search Input UI



Search filtering messages

```
{messages
    .filter((msg) =>
        msg.content?.toLowerCase().includes(searchChat.toLowerCase()) ||
        msg.name?.toLowerCase().includes(searchChat.toLowerCase))
    )
```

Example screenshot

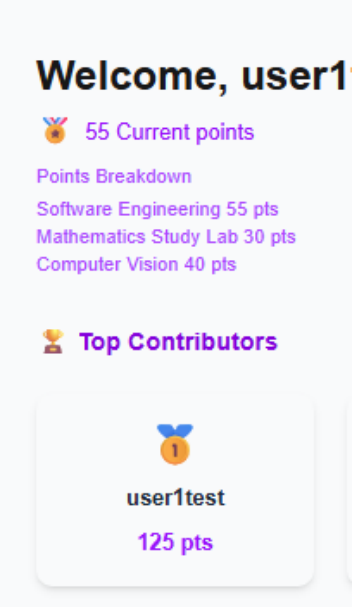


I had to fix an issue with user breakdown displaying incorrect data after viewing leaderboard user

So, i separated state into two variables

```
const [myBreakdown, setMyBreakdown] = useState(null);
const [selectedUserBreakdown, setSelectedUserBreakdown] = useState(null);
```

Separated total points from current points to remove confusion for users' current points is points they can use to redeem a gift item and in leaderboard it shows



Added dynamic progress bar to reward system

Calculated next available reward based on the user points

Visualised progress using percentage-based UI

Improved user motivation with goal tracking

```
const nextReward =
  rewards
    .filter(r => r.cost > points)
    .sort((a, b) => a.cost - b.cost)[0]?.cost || 100;

const progress = Math.min((points / nextReward) * 100, 100);
```

```
  {rewards.length > 0 && (
    <div className="mt-2">
      <div className="text-xs text-gray-500 mb-1">
        Progress to next reward ({nextReward} pts)
      </div>

      <div className="w-full bg-gray-200 rounded-full h-2">
        <div
          className="bg-purple-600 h-2 rounded-full transition-all duration-500"
          style={{ width: `${progress}%` }}
        />
      </div>
    </div>
  )}
</div>
```

Implemented a dynamic reward availability indicator on the dashboard to provide immediate visual feedback on how many rewards a user can redeem

Calculated redeemable rewards by comparing user points against reward costs, ensuring accurate and real time eligibility tracking

```
const [redeemableCount, setRedeemableCount] = useState(0);
```

```
useEffect(() => {
  if (!user?.email) return;

  Promise.all([
    fetch(`/api/getUserPoints?email=${user.email}`).then(res => res.json()),
    fetch("/api/getRewards").then(res => res.json())
  ]).then(([pointsData, rewardsData]) => {

    const userPoints = pointsData.points || 0;
    const rewards = rewardsData.rewards || [];

    const count = rewards.filter(r => userPoints >= r.cost).length;

    setRedeemableCount(count);
  });

}, [user]);
```

Added a notification style badge UI to improve visibility, inspired by common app design patterns

The notification system was enhanced by integrating real-time updates with persistent storage and improved state management, ensuring reliability across navigation and page reloads.

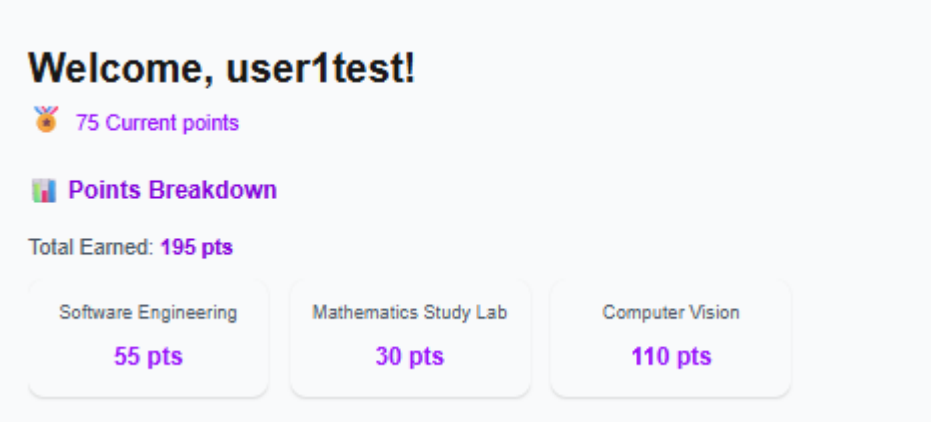
```
useEffect(() => {
  const storedNotifications = localStorage.getItem("notifications");
  const storedUnreadCount = localStorage.getItem("unreadCount");

  if (storedNotifications) {
    setNotifications(JSON.parse(storedNotifications));
  }

  if (storedUnreadCount) {
    setUnreadCount(Number(storedUnreadCount));
  }

  hasLoadedNotifications.current = true;
}, []);
```

```
onClick={() => {
  setUnreadCount(0);
  setShowNotifications(!showNotifications);
}}
```

I enabled individual notification interaction which allowed users to click a notification and remove only that specific item which prevents all notifications from resetting and improves accuracy and aligns with real world notification behaviour

```
setNotifications(prev => {
  const updated = prev.filter((_, i) => i !== index);

  localStorage.setItem("notifications", JSON.stringify(updated));

  return updated;
});
```

30/03/2026 - 05/03/2026



Enhanced the UI of the points breakdown to a more visual representation compared to the previous version

Created the api pinMessage responsible for handling the message pinning functionality.

JS pinMessage.js U

Updates the selected message in the database by marking it as pinned

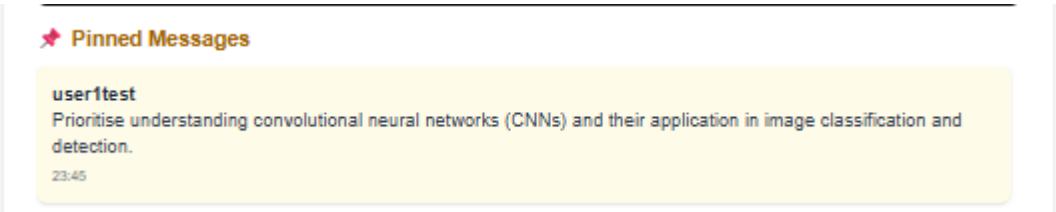
```
const result = await db.collection("spaceMessages").updateOne(
    { _id: new ObjectId(messageId) },
    { $set: { pinned: true } }
);
```

```
{messages.some((msg) => msg.pinned) && (
    <div className="mb-4">
        <h4 className="font-semibold text-yellow-700 mb-2">
            📌 Pinned Messages
        </h4>
```

This enables the space leader to pin important messages, ensuring key information remains visible to all participants

```
{isLeader && (
    <button
        onClick={async () => {
            const res = await fetch(
                "/api/pinMessage",
                {
                    method: "POST",
                    headers: {
                        "Content-Type":
                            "application/json",
                    },
                    body: JSON.stringify({
                        messageId: msg._id,
                    }),
                }
            );
```

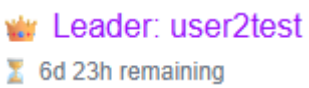
Message pinned in the discussion section of the space



A time-based leader rotation indicator was introduced to provide users with real-time visibility and clarity of leadership duration

```
const getTimeRemaining = (lastRotation) => {
```

```
<p className="text-lg text-gray-500 mt-1">
    ⏳ {getTimeRemaining(space.lastRotation)}
</p>
```



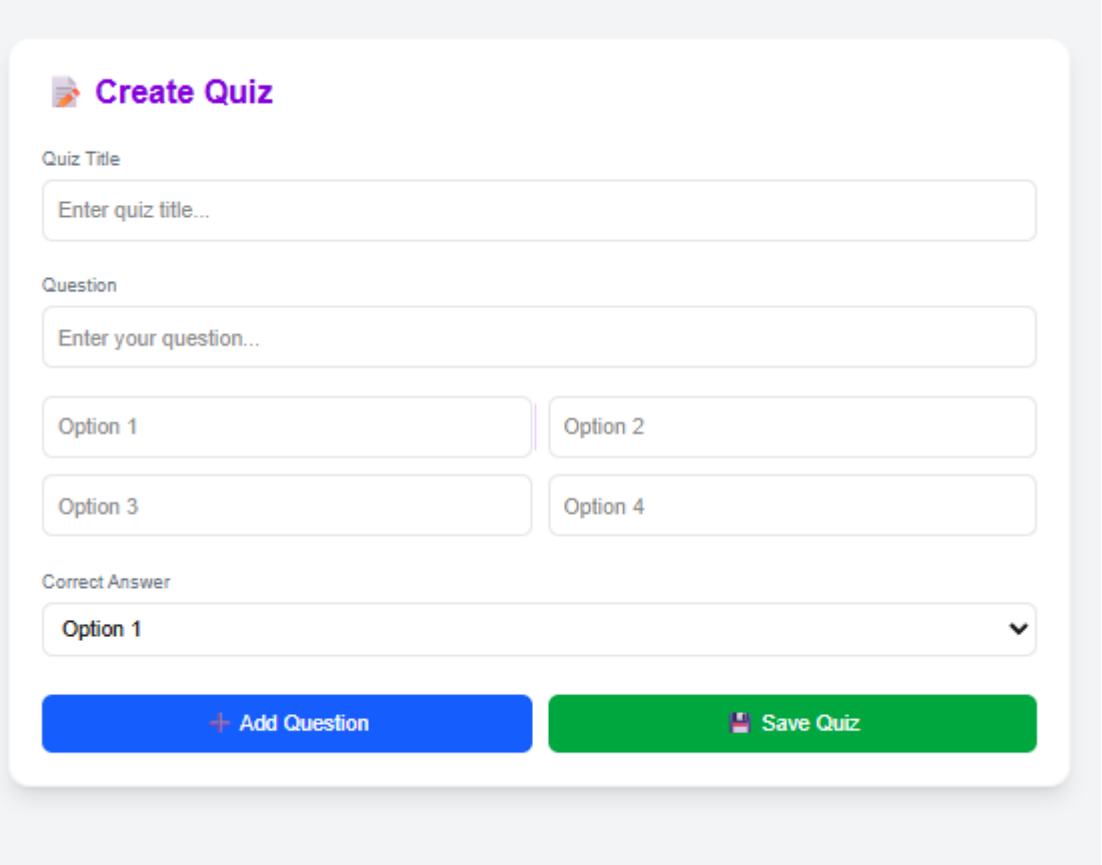
Leader gets rewarded for creating up to 3 quizzes which is the minimum requirement and points reduction for creating less than 3 quizzes

```
if (currentLeader) {
    let pointsChange = 0;

    if (quizCount >= 3) {
        pointsChange = 40;
    } else {
        pointsChange = -20;
    }

    await db.collection("users").updateOne(
        { email: currentLeader },
        {
            $inc: { points: pointsChange }
        }
    );
}
```

Added ?spaceId= &{id}` to ensure that the rotation leadership duration was assigned to the specific space to avoid consistency issues

```
fetch(`"/api/rotateLeader"?spaceId=${id}`, { method: "POST" });
```

The quiz creation interface was enhanced with improved layout, visual hierarchy and user interaction feedback to provide a more intuitive experience.



06/04/2026 - 12/04/2026

Created getSpaceLeaderboard API to allow users in a particular space to view the current leaderboard points

```
JS getSpaceLeaderboard.js U
```

This adds quiz scores only for that space

```
const quizAttempts = await db
    .collection("quizAttempts")
    .find({ userEmail: member.email })
    .toArray();

for (const attempt of quizAttempts) {
    const quizId = attempt.quizId;

    const quizSpace = await db.collection("spaces").findOne({
        "quizzes._id": new ObjectId(quizId),
    });

    if (!quizSpace) continue;

    if (quizSpace._id.toString() === space._id.toString()) {
        spacePoints += attempt.score;
    }
}
```

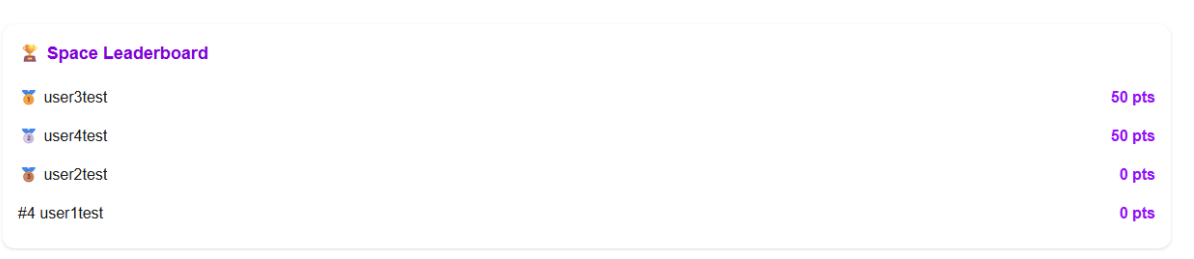
Implemented a state variable to store and manage space-specific leaderboard data dynamically within the component.

```
const [spaceLeaderboard, setSpaceLeaderboard] = useState([]);
```

Displayed user identifiers alongside rankings to enhance clarity and readability of leaderboard entries and integrated a formatted points display to visually highlight user performance within the space.

```
    {spaceLeaderboard.map((u, i) => (
        <div key={u.email} className="flex justify-between py-2">
            <span>
                {i === 0 ? "🥇" : i === 1 ? "🥈" : i === 2 ? "🥉" : `#${i + 1}`}{" "}
                {u.name}
            </span>
            <span className="text-purple-600 font-semibold">
                {u.points} pts
            </span>
        </div>
    ))}
</div>
```

Space Leaderboard UI



Implementing Reply feature in Discussion Section

```
const [replyingTo, setReplyingTo] = useState(null);
```

```
await collection.insertOne({
    spaceId,
    sender,
    name,
    type: type || "text",
    content: content || message,
    timestamp,
    replyTo: req.body.replyTo || null
});
```

Added replyTo: replyingTo || null

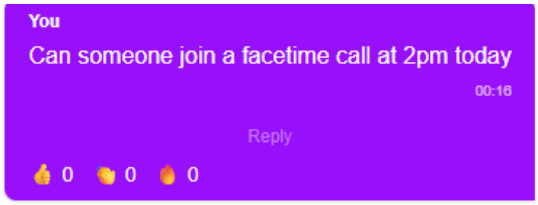
```
const msg = {
    spaceId: id,
    spaceName: space.title,
    sender: currentUserEmail,
    name: currentUserName,
    type: detectMessageType(chatInput.trim()),
    content: chatInput.trim(),
    timestamp: Date.now(),
    replyTo: replyingTo || null
};
```

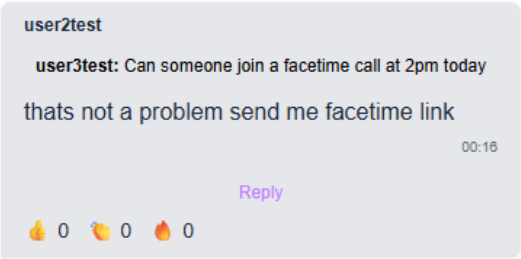
```
_id: ObjectId('69d8333e6998078fe5ab121a')
spaceId : "69bd6a9548b2522a40619a15"
sender : "user3test@gmail.com"
name : "user3test"
type : "text"
content : "Can someone join a facetime call at 2pm today"
timestamp : 1775776574523
replyTo : null
```

```
_id: ObjectId('69d833506998078fe5ab121b')
spaceId : "69bd6a9548b2522a40619a15"
sender : "user2test@gmail.com"
name : "user2test"
type : "text"
content : "thats not a problem send me facetime link"
timestamp : 1775776592914
replyTo : "69d8333e6998078fe5ab121a"
```

Reply preview that displays the parent messager

```
{/*  REPLY PREVIEW */}
{parentMessage && (
    <div className="bg-gray-200 text-black text-xs p-2 rounded mb-1">
        <strong>{parentMessage.name || "User"}:</strong>{" "}
        {parentMessage.content}
    </div>
)}
```





Delete Message Functionality

```
const [openMenu, setOpenMenu] = useState(null);
```

```
JS deleteMessage.js                    U
```

```
const handleDelete = async (messageId) => {
    try {
        const res = await fetch("/api/deleteMessage", {
            method: "POST",
            headers: {
                "Content-Type": "application/json",
            },
            body: JSON.stringify({
                messageId,
                userEmail: currentUserEmail
            }),
        });
```

```
{openMenu === msg._id && (
    <div className="absolute right-0 mt-1 bg-white shadow rounded text-xl
        <button
            onClick={() => handleDelete(msg._id)}
            className="block px-3 py-2 hover:bg-gray-100 text-red-500"
        >
            Delete
        </button>
```